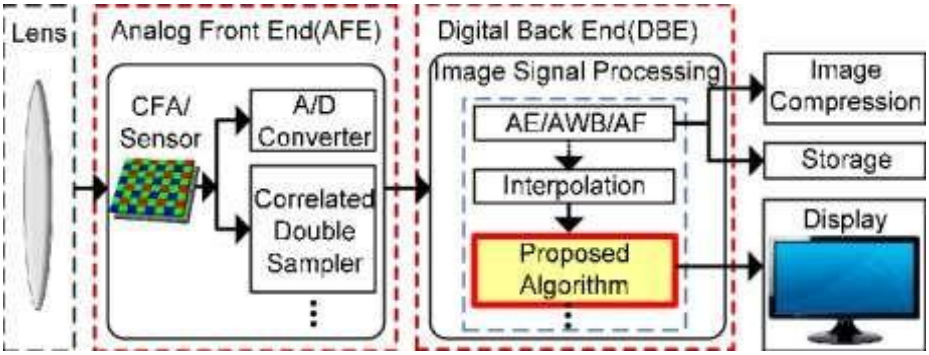
UNIT I

INTRODUCTION

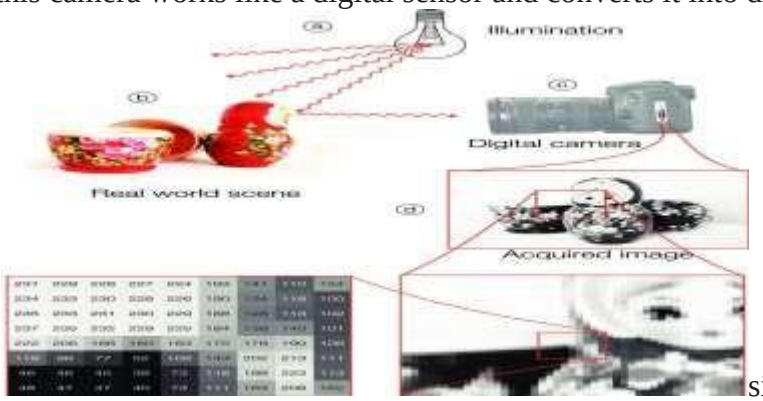
Image Processing, Computer Vision, What is Computer Vision - Low-level, Mid-level, High-level; Fundamentals of Image Formation, Transformation: Orthogonal, Euclidean, Affine, Projective, Fourier Transform, Convolution in OpenCV and Filtering, Image Enhancement, Restoration, Histogram Processing.

1.1 INTRODUCTION :

A digital image is formed by the small bits of data i.e. pixels, which are stored in



computers. When we capture an image in our digital camera in presence of light then this camera works like a digital sensor and converts it into digital



signals.

1.2 IMAGE PROCESSING

1.2 Introduction

Digital Image Processing means processing digital image by means of a digital computer. We can also say that it is a use of computer algorithms, in order to get enhanced image either to extract some useful information.
Digital image processing is the use of algorithms and mathematical models to process and analyze digital images. The goal of digital image processing is to enhance the quality of images, extract meaningful information from images, and automate image-based tasks.

The basic steps involved in digital image processing are:

1. Image acquisition: This involves capturing an image using a digital camera or scanner, or importing an existing image into a computer.
2. Image enhancement: This involves improving the visual quality of an image, such as increasing contrast, reducing noise, and removing artifacts.
3. Image restoration: This involves removing degradation from an image, such as blurring, noise, and distortion.
4. Image segmentation: This involves dividing an image into regions or segments, each of which corresponds to a specific object or feature in the image.
5. Image representation and description: This involves representing an image in a way that can be analyzed and manipulated by a computer, and describing the features of an image in a compact and meaningful way.
6. Image analysis: This involves using algorithms and mathematical models to extract information from an image, such as recognizing objects, detecting patterns, and quantifying features.
7. Image synthesis and compression: This involves generating new images or compressing existing images to reduce storage and transmission requirements.
8. Digital image processing is widely used in a variety of applications, including medical imaging, remote sensing, computer vision, and multimedia.

1.2.1 Types of an image

1. **BINARY IMAGE**– The binary image as its name suggests, contain only two pixel elements i.e 0 & 1,where 0 refers to black and 1 refers to white. This image is also known as Monochrome.
2. **BLACK AND WHITE IMAGE**– The image which consist of only black and white color is called BLACK AND WHITE IMAGE.
3. **8 bit COLOR FORMAT**– It is the most famous image format.It has 256 different shades of colors in it and commonly known as Grayscale Image. In this format, 0 stands for Black, and 255 stands for white, and 127 stands for gray.
4. **16 bit COLOR FORMAT**– It is a color image format. It has 65,536 different colors in it.It is also known as High Color Format. In this format the distribution of color is not as same as Grayscale image.

1.2.2 PHASES OF IMAGE PROCESSING:

1. **ACQUISITION**– It could be as simple as being given an image which is in digital form. The main work involves:
2. Scaling
3. Color conversion(RGB to Gray or vice-versa)
4. **IMAGE ENHANCEMENT**– It is amongst the simplest and most appealing in areas of Image Processing it is also used to extract some hidden details from an image and is subjective.
5. **IMAGE RESTORATION**– It also deals with appealing of an image but it is objective(Restoration is based on mathematical or probabilistic model or image degradation).
6. **COLOR IMAGE PROCESSING**– It deals with pseudocolor and full color image processing color models are applicable to digital image processing.
7. **WAVELETS AND MULTI-RESOLUTION PROCESSING**– It is foundation of representing images in various degrees.
8. **IMAGE COMPRESSION**-It involves in developing some functions to perform this operation. It mainly deals with image size or resolution. 7.**MORPHOLOGICAL PROCESSING**-It deals with tools for extracting image components that are useful in the representation & description of shape.

9. **SEGMENTATION PROCEDURE-**It includes partitioning an image into its constituent parts or objects. Autonomous segmentation is the most difficult task in Image Processing.
10. **REPRESENTATION & DESCRIPTION-**It follows output of segmentation stage, choosing a representation is only the part of solution for transforming raw data into processed data.
11. **OBJECT DETECTION AND RECOGNITION-**It is a process that assigns a label to an object based on its descriptor.

1.3 Computer Vision

Computer vision is concerned with modeling and replicating human vision using computer software and hardware. Formally if we define computer vision then its definition would be that computer vision is a discipline that studies how to reconstruct, interrupt and understand a 3d scene from its 2d images in terms of the properties of the structure present in scene.

It needs knowledge from the following fields in order to understand and stimulate the operation of human vision system.

- Computer Science
- Electrical Engineering
- Mathematics
- Physiology
- Biology
- Cognitive Science

1.3.1 Computer Vision Vs Image Processing

Image processing studies image to image transformation. The input and output of image processing are both images.

Computer vision is the construction of explicit, meaningful descriptions of physical objects from their image. The output of computer vision is a description or an interpretation of structures in 3D scene.

Example Applications

- Robotics
- Medicine
- Security
- Transportation

- Industrial Automation

Robotics Application

- Localization-determine robot location automatically
- Navigation
- Obstacles avoidance
- Assembly (peg-in-hole, welding, painting)
- Manipulation (e.g. PUMA robot manipulator)
- Human Robot Interaction (HRI): Intelligent robotics to interact with and serve people

Medicine Application

- Classification and detection (e.g. lesion or cells classificationand tumor detection)
- 2D/3D segmentation
- 3D human organ reconstruction (MRI or ultrasound)
- Vision-guided robotics surgery

Industrial Automation Application

- Industrial inspection (defect detection)
- Assembly
- Barcode and package label reading
- Object sorting
- Document understanding (e.g. OCR)

1.3.2 Computer Graphics

Computer graphics are graphics created using computers and the representation of image data by a computer specifically with help from specialized graphic hardware and software. Formally we can say that Computer graphics is creation, manipulation and storage of geometric objects (modeling) and their images (Rendering).

3d Animation

- Used heavily in the movie industry by companies such asPixar, DresmsWorks
- To add special effects in games and movies.

1.3.3 Levels in Computer Vision:

Computer Vision can be thought of as a hierarchy of processing levels, with each level building on the previous one to extract more and more complex information from the input image. The main levels in Computer Vision are:

1. Low-level processing: This level involves basic operations such as image

filtering, edge detection, and feature extraction. These techniques are used to preprocess the input image and enhance its quality.

2. Mid-level processing: This level involves extracting more complex features from the image, such as contours, shapes, and textures. These features are used to represent the image in a more meaningful way and can be used for tasks such as object recognition and segmentation.
3. High-level processing: This level involves using the features extracted at the mid-level to perform more complex tasks such as object detection, scene understanding, and image classification. This level often involves the use of machine learning algorithms such as deep neural networks.

Overall, the levels in Computer Vision build on each other to extract increasingly complex information from the input image, ultimately leading to a more comprehensive understanding of the visual world.

1.3.4 Low-level computer vision

- Low-level computer vision is concerned with the basic processing of images and video. This includes tasks such as:
- Image filtering: Smoothing, sharpening, and edge detection
- Feature extraction: Identifying corners, edges, and other features in images
- Image segmentation: Grouping pixels together based on similarity
- Optical flow: Tracking the motion of objects in video
- Low-level computer vision tasks are typically performed on raw pixel data. They are often used as preprocessing steps for more complex tasks, such as object recognition and scene understanding.

1.3.5Mid-level computer vision

- Mid-level computer vision is concerned with the interpretation of images and video. This includes tasks such as:
- Object detection: Identifying objects in images and video
- Object recognition: Classifying objects into different categories
- Scene understanding: Understanding the overall meaning of a scene
- Mid-level computer vision tasks typically use features extracted from low-level processing to make inferences about the content of images and video.

1.3.6High-level computer vision

- High-level computer vision is concerned with the application of computer vision to real-world problems. This includes tasks such as:
    - Image-based retrieval: Finding images similar to a query image
    - Action recognition: Identifying actions in video
    - Medical image analysis: Analyzing medical images for diagnosis and treatment

- High-level computer vision tasks typically use a combination of mid-level and low-level processing to solve complex problems.

1.4 FUNDAMENTALS OF IMAGE FORMATION

Image formation is an analog to digital conversion of an image with the help of 2D Sampling and Quantization techniques that is done by the capturing devices like cameras. In general, we see a 2D view of the 3D world.

In the same way, the formation of the analog image took place. It is basically a conversion of the 3D world that is our analog image to a 2D world that is our Digital image.

Imaging:

The mapping of a 3D world object into a 2D digital image plane is called *imaging.* In order to do so, each point on the 3D object must correspond to the image plane. We all know that light reflects from every object that we see thus enabling us to capture all

1.5 Transformation: Orthogonal

Transformation is a function. A function that maps one set to another set after performing some operations.

An image is obtained in spatial coordinates (x, y) or (x, y, z). There are many advantages if the spatial domain image is transformed into another domain. In which solution of any problem canbe found easily.

Following are two types of transformations:

1. Fourier Transform

Fourier transform is mainly used for image processing. In the Fourier transform, the intensity of the image is transformed into frequency variation

and then to the frequency domain. It is used for slow varying intensity images such as the background of a passport size photo can be represented as low-frequency components and the edges can be represented as high-frequency components. Low-frequency components can be removed using filters of FT domain. When an image is filtered in the FT domain, it contains only the edges of the image. And if we do inverse FT domain to spatial domain then also an image contains only edges. Fourier transform is the simplest technique in which edges of the image can be fined.

Two Dimensional Fourier Transform

$$v(k,l)=\sum_{m=0}^{N-1}\sum_{n=0}^{N-1}u(m,n)W_N^{km}W_N^{ln},\quad W_N \triangleq \exp\left(\frac{-j2\pi}{N}\right)$$

$$u(m,n)=\frac{1}{N^2}\sum_{k=0}^{N-1}\sum_{l=0}^{N-1}v(k,l)W_N^{-km}W_N^{-ln}$$

$$\left.\begin{aligned} v(k,l)&=\frac{1}{N}\sum_{m=0}^{N-1}\sum_{n=0}^{N-1}u(m,n)W_N^{km}W_N^{ln} \\ u(m,n)&=\frac{1}{N}\sum_{k=0}^{N-1}\sum_{l=0}^{N-1}v(k,l)W_N^{-km}W_N^{-ln} \end{aligned}\right\}\text{Unitary DFT Pair}$$

Matrix Notation:

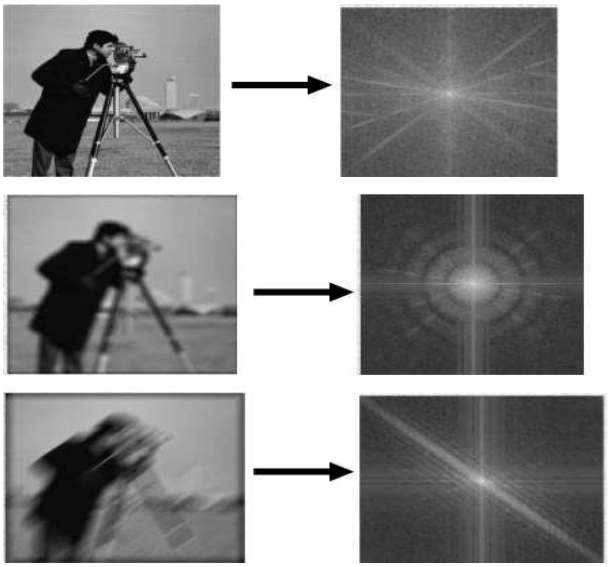
$$\mathbf{V}=\mathbf{FUF}\Leftrightarrow\mathbf{U}=\mathbf{F}^*\mathbf{VF}^*$$

$$\mathbf{F}=\left\{\frac{1}{\sqrt{N}}W_N^{kn}\right\}_{k,n=0}^{N-1}$$

Properties of Fourier transformation are as follows:

- Symmetric Unitary
- Periodic Extension
- Sampled Fourier
- Fast
- Conjugate Symmetry
- Circular Convolution

Example of Blurred image and its Fourier transformation



1.5.1 Discrete Cosine Transformation (DCT)

In Discrete Cosine Transformation, coefficients carry information about the pixels of the image.

$$\mathbf{C} = \{c(k,n)\}$$

$$c(k,n) = \begin{cases} \dfrac{1}{\sqrt{N}} & k = 0, 0 \le n \le N-1 \\ \dfrac{2}{\sqrt{N}} \cos\left(\dfrac{\pi(2n+1)k}{2N}\right) & 1 \le k \le N-1, 0 \le n \le N-1 \end{cases}$$

$$v(k) = \alpha(k) \sum_{n=0}^{N-1} u(n) \cos\left(\frac{\pi(2n+1)k}{2N}\right), \quad 0 \le k \le N-1$$

$$\alpha(0) \triangleq \frac{1}{\sqrt{N}}, \quad \alpha(k) \triangleq \frac{2}{\sqrt{N}}, \quad 1 \le k \le N-1$$

$$u(n) = \sum_{k=0}^{N-1} \alpha(k) v(k) \cos\left(\frac{\pi(2n+1)k}{2N}\right), \quad 0 \le n \le N-1$$

Two Dimension Discrete cosine transformations:

$$- \mathbf{A=A^{*}=C}$$

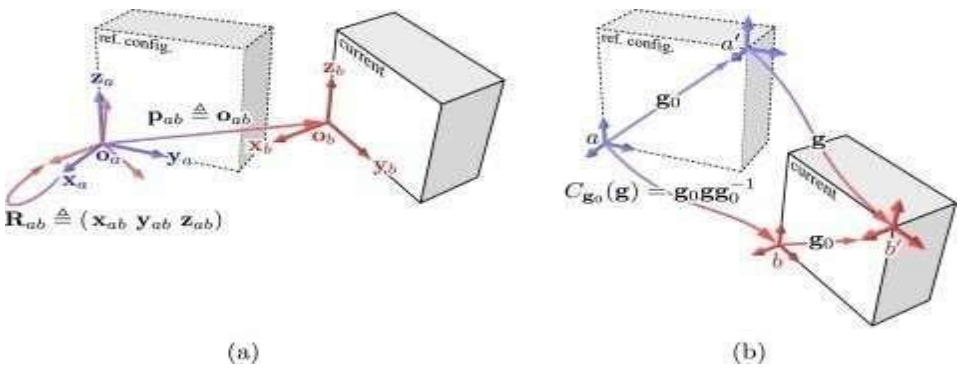
Properties of Discrete cosine transformation are as following:

- Real and Orthogonal: C=C* → C-1=CT
- Not! Real part of DFT
- Fast Transform
- Excellent Energy Compaction (Highly Correlated Data)

1.5.2 Orthogonal transforms:

Orthogonal transforms are used in a number of image processing operations. As an example, a particular orthogonal transform is the Discrete Cosine Transform The DCT is a key element in image compression.

Euclidean :

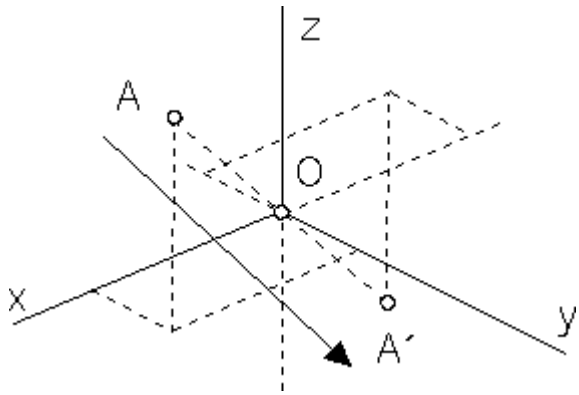


1.5.3 Euclidean transformations:

Euclidean - metric transformations preserve invariant size of the figure, they change only the figure position in the space. Relation between coordinates of the original and its image in the giventransformation is expressed by equations of the transformation. Matrix of a linear transformation is the matrix of this

$A' = A \,.T_{Sz}$

Symmetry with respect to the origin *O*



$$\mathbf{T}_{SO} = \begin{pmatrix} -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

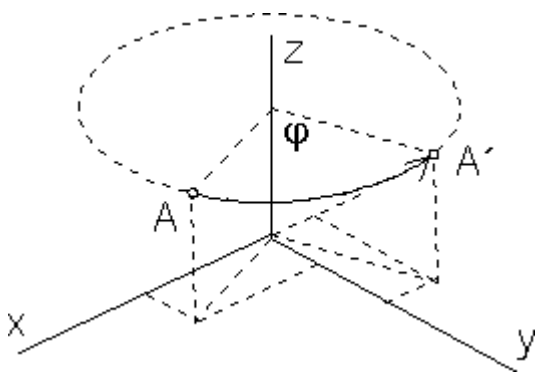
$x' = -x$

$y' = -y$

$z' = -z$

$A' = A \,.T_{SO}$

Revolution about the coordinate axis *z* by angle j



1.12 Introduction

$$T_{Oz} = \begin{pmatrix} \cos\varphi & \sin\varphi & 0 & 0 \\ -\sin\varphi & \cos\varphi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$x' = x \cos j - y \sin j$

$y' = x \sin j + y \cos j$

$z' = z$

$A' = A \,.T_{Oz}$

Translation by vector (m, n, p, 0)

$$\mathbf{T}_{PS_\pi} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & c & 0 & 1 \end{pmatrix}$$

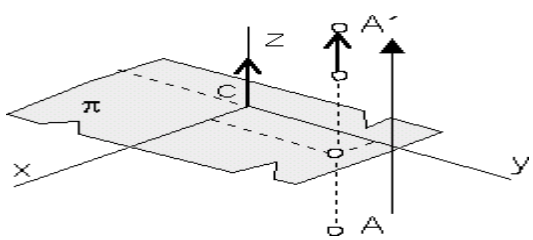
$x' = x + m$

$y' = y + n$

$z' = z + p$

$A' = A \,.T_P$

Symmetry with respect to the coordinate plane p translated by vector (0, *c*, 0, 0)



$$\mathbf{T}_{OSv} = \begin{pmatrix} -1 & 0 & 0 & 0 \\ 0 & \cos\varphi & \sin\varphi & 0 \\ 0 & -\sin\varphi & \cos\varphi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$x' = -x$

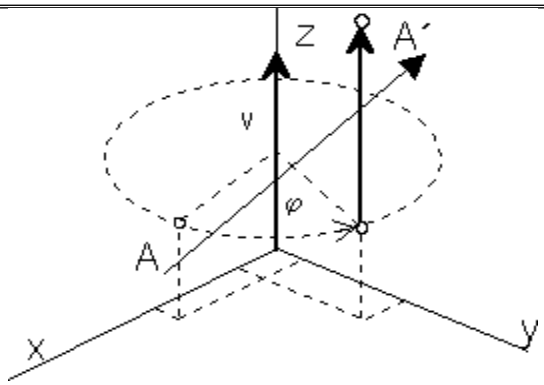
$y' = y \cos j - z \sin j$

$z' = y \sin j + z \cos j$

$A' = A \,.\, T_{OSn}$

Helical movement about the coordinate axis *z* with the translation vector (0,0,*v*,0) related to the revolution angle j

---



$$\mathbf{T}_{S} = \begin{pmatrix} \cos\varphi & \sin\varphi & 0 & 0 \\ -\sin\varphi & \cos\varphi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & v & 1 \end{pmatrix}$$

$x' = x \cos j - y \sin j$

$y' = x \sin j + y \cos j$

1.14 Introduction

$z' = z + v$

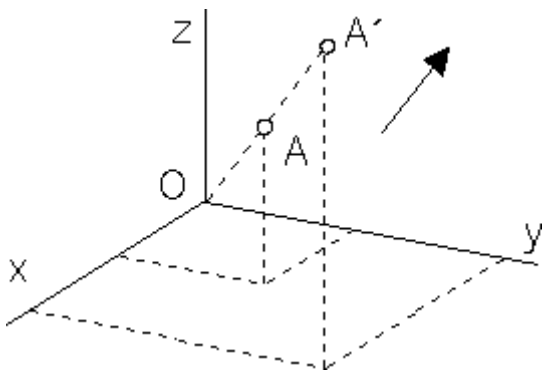
$A' = A \cdot T_S$

Determinant of the matrix T_M of any metric transformation has the value ±1, it is valid $|T_M|^2 = 1$.

1.5.4 Affine transformations of the space $_\infty E^3$

Affine transformations do not preserve invariant the size of line segments and angles. Special subset of affine transformations form similarities, preserving invariant the size of angles. Any Euclidean transformation is an affine transformation (similarity) $_\infty E^3$

Scaling to the centre *O*with the nonzero coefficient *s* (scaleon all coordinate axes)



1.5.5 Projective transformations of the space $_\infty E^3$

Projective transformation of the extended Euclidean space E^3_∞ is any linear geometric transformationof the space called collineation.

In addition to incidence, collinear transformations preserve invariant ration of four points on a line.

All Euclidean and affine transformations are collineations with special properties and invariantelements.

General projective transformation

$$\mathbf{T}_K = \begin{pmatrix} a & b & c & q \\ d & e & f & r \\ g & k & i & t \\ m & n & p & s \end{pmatrix}$$

$s \neq 0$, $|T_K| \neq 0$, matrix of the system of equations

$$x' = ax + dy + gz + m$$
$$y' = bx + ey + kz + n$$
$$z' = cx + fy + iz + p$$
$$h' = qx + ry + tz + s$$

$$A = (x, y, z, 1) \rightarrow A' = (x', y', z', h') = h'\left(\frac{x'}{h'}, \frac{y'}{h'}, \frac{z'}{h'}, 1\right)$$

1.5.6 Convolution Filtering

Convolution is a simple mathematical operation which is fundamental to many common image processing operators. Convolution provides a way of `multiplying together' two arrays of numbers, generally of different sizes, but of the same dimensionality, to produce a third array of numbers of the same dimensionality.

Simple box blur

Here's a first and simplest. This convolution kernel has an averaging effect. So you end up with a slight blur. The image convolution kernel is:

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

Note that the sum of all elements of this matrix is 1.0. This is important. If the sum is not exactly one, theresultant image will be brighter or darker.

The laplacian operator

The laplacian is the second derivative of the image. It is extremely sensitive to noise, so it isn't used as much as other operators. Unless, of course you have specific requirements.

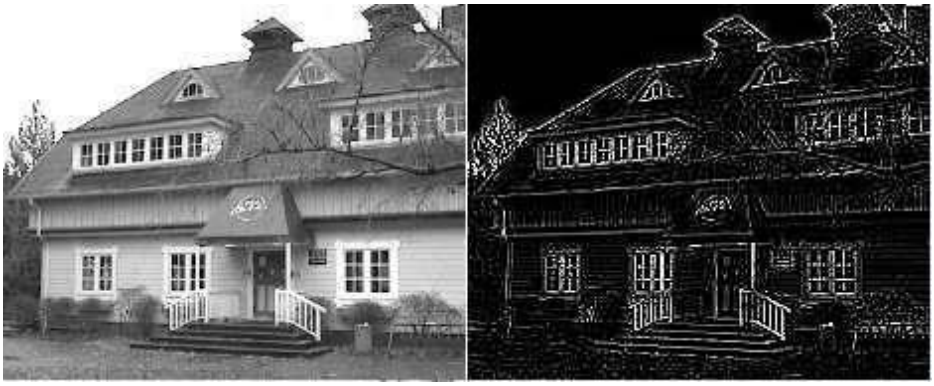
0	-1	0
-1	4	-1
0	-1	0

The laplacian operator

-1	-1	-1
-1	8	-1
-1	-1	-1

The laplacian operator (include diagonals)

Here's the result with the convolution kernel without diagonals:



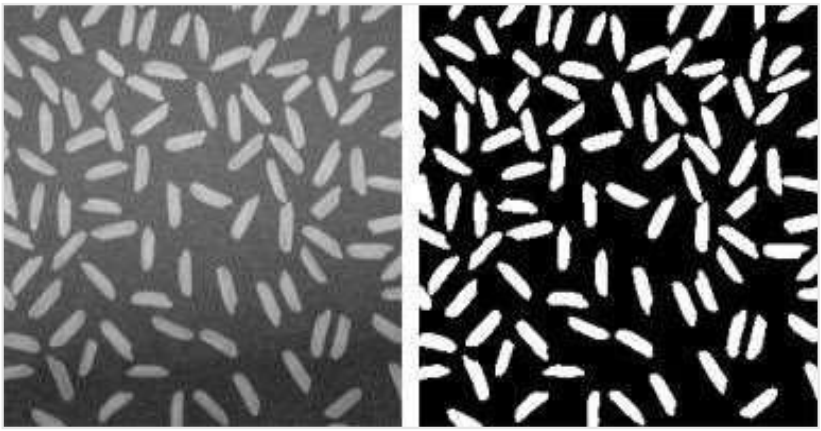
1.6 Image enhancement Restoration

Image enhancement is the process of adjusting digital images so that the results are more suitable for display or further image analysis. For example, you can remove noise, sharpen, or brighten an image, making it easier to identify key features.

Here are some useful examples and methods of image enhancement:

- Filtering with morphological operators
- Histogram equalization
- Noise removal using a Wiener filter
- Linear contrast adjustment
- Median filtering
- Unsharp mask filtering
- Contrast-limited adaptive histogram equalization (CLAHE)
- Decorrelation stretch

The following images illustrate a few of these examples:



Correcting nonuniform illumination with morphological operators.

Enhancing grayscale images with histogram equalization

<u>Deblurring images using a Wiener filter.</u>

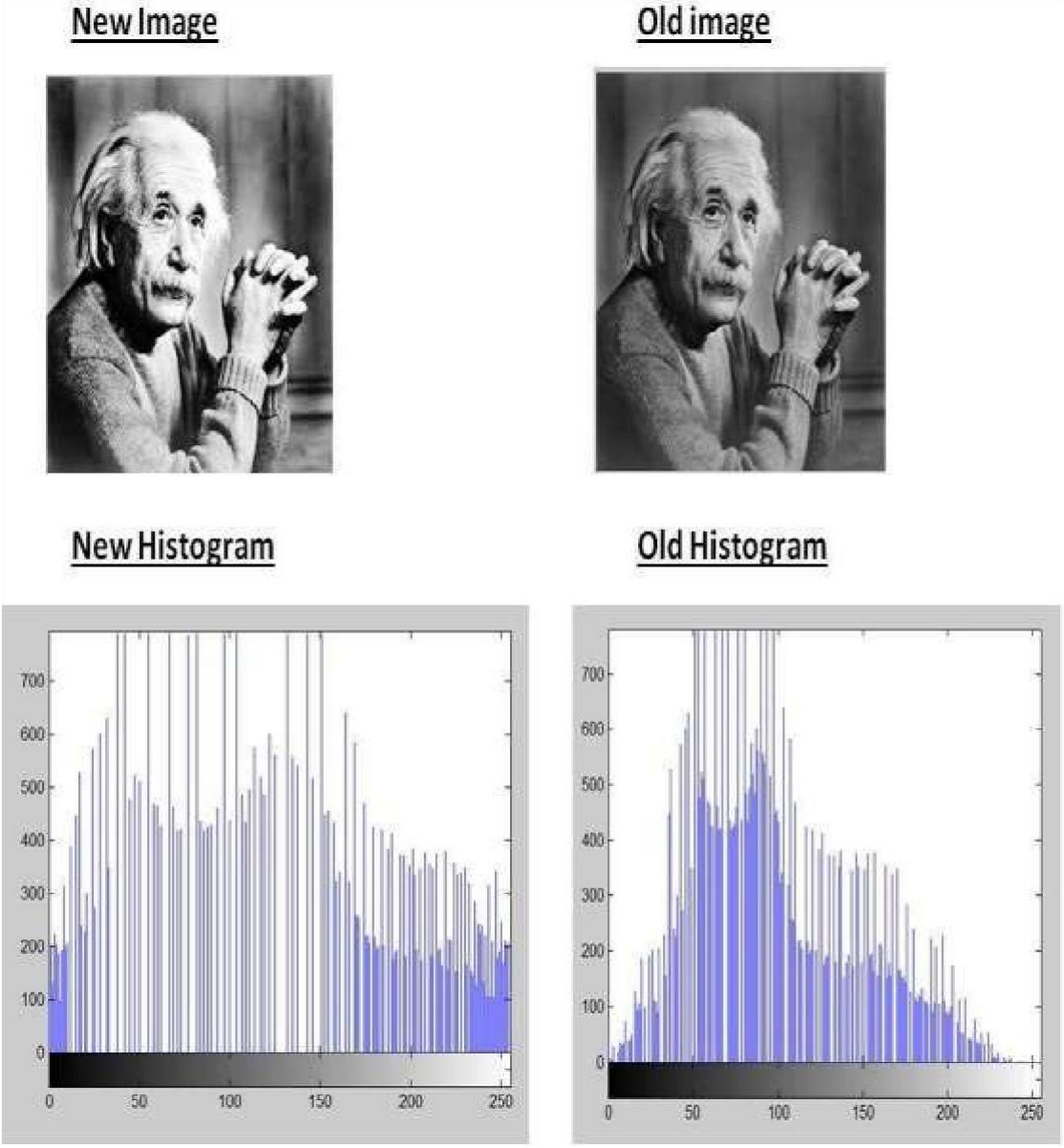
Image enhancement algorithms include deblurring, filtering, and contrast methods.

1.7 Restoration

Restoration process improves the appearance of the image. The degraded image is the convolution of the original image, degraded function, and additive noise. The process of restoration is deconvolved this degraded image to obtain noiselessly and deblurred original image.

1.8 Histogram Processing

Histogram Equalization is a computer image processing technique used to improve contrast in images . It accomplishes this by effectively spreading out the most frequent intensity values, i.e. stretching out the intensity range of the image.



In digital image processing, the histogram is used for graphical representation of a digital image. A graph is a plot by the number of pixels for each tonal value. Nowadays, image histogram is present in digital cameras. Photographers use them to see the distribution of tones captured.

In a graph, the horizontal axis of the graph is used to represent tonal variations whereas the vertical axis is used to represent the number of pixels in that particular pixel. Black

and dark areas are represented in the left side of the horizontal axis, medium grey color is represented in the middle, and the vertical axis represents the size of the area.

1.8.1 Histogram Processing Techniques

Histogram Sliding

In Histogram sliding, the complete histogram is shifted towards rightwards or leftwards. When a histogram is shifted towards the right or left, clear changes are seen

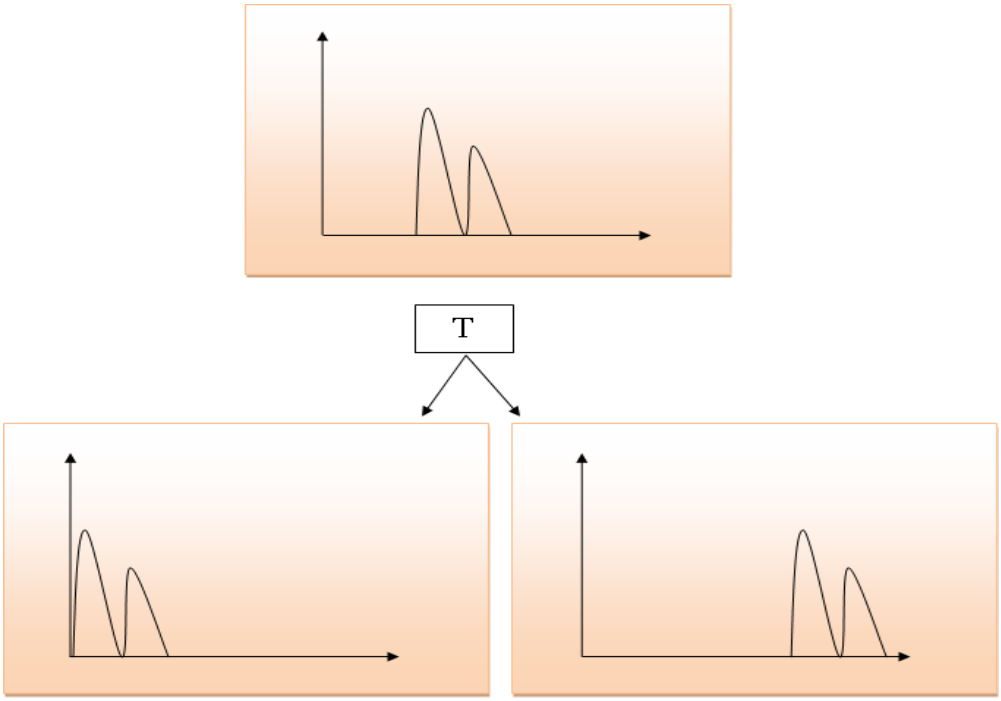


Fig. Histogram Sliding

in the brightness of the image. The brightness of the image is defined by the intensity of light which is emitted by a particular light source.

UNIT II

FEATURE EXTRACTION AND FEATURE SEGMENTATION

Feature Extraction -Edges - Canny, LOG, DOG; Line detectors (Hough Transform), Corners - Harris and Hessian Affine, Orientation Histogram, SIFT, SURF, HOG, GLOH, Scale-Space Analysis- Image Pyramids and Gaussian derivative filters, Gabor Filters and DWT. Image Segmentation -Region Growing, Edge Based approaches to segmentation, Graph-Cut, Mean-**Shift, MRFs, Texture**

2.1 Feature Extraction in Computer Vision

Feature extraction is important for several reasons:

- Reduces data dimensionality: Images contain a large amount of data, which can make them computationally expensive to process. Feature extraction can reduce the dimensionality of the data by selecting only the most relevant features, which can improve the efficiency of computer vision algorithms.
- Improves accuracy and robustness: Feature extraction can improve the accuracy and robustness of computer vision algorithms by removing noise and irrelevant information from the data. This can help to improve the performance of algorithms in challenging conditions, such as low-light or noisy images.

- Enables better feature representation: Feature extraction can be used to create more meaningful and compact representations of images. These representations can then be used to improve the performance of machine learning algorithms.

Types of feature extraction

There are many different types of feature extraction algorithms, but they can be broadly categorized into two main types:

- Handcrafted features: These features are manually designed by experts and are based on prior knowledge of the domain.
- Learned features: These features are automatically learned from data using machine learning algorithms.

Examples of feature extraction

- Here are a few examples of commonly used feature extraction techniques:
- Edge detection: Edge detection algorithms identify sharp discontinuities in intensity in an image. Edges are important features in images because they often correspond to object boundaries.
- Corner detection: Corner detection algorithms identify points in an image where two edges intersect. Corners are important features in images because they often correspond to object corners.
- Region-based feature extraction: Region-based feature extraction algorithms divide an image into regions and then extract features from each region. This is a common approach for object detection and image segmentation.
- Texture analysis: Texture analysis algorithms extract information about the texture of an image. Texture is an important feature in images because it can be used to classify objects or surfaces.

2.2 Edges - Canny, LOG, DOG

Edge detection: The Canny edge detection algorithm is a popular edge detection algorithm that uses a multi-stage approach to identify edges. The first stage of the Canny algorithm applies a Gaussian blur to the image to remove noise. The second stage of the algorithm applies the Sobel operator to the image to compute the magnitude and direction of the gradient at each pixel. The third stage of the algorithm applies non-maximum suppression to the gradient magnitude image to thin the edges and remove spurious pixels. Finally, the fourth stage of the algorithm applies hysteresis to the gradient magnitude image to threshold the edges and remove weak edges.

ORIGINAL IMAGE

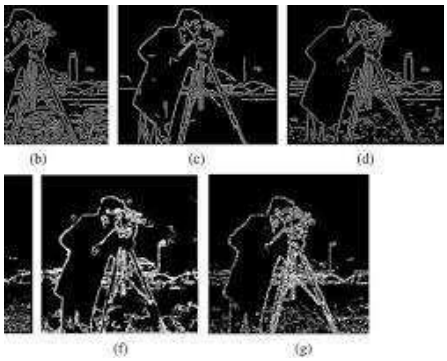


IMAGE AFTER CANNY EDGE DETECTION

- **Corner detection**: The Harris corner detection algorithm is a popular corner detection algorithm that uses the second moment matrix to identify corners. The Harris corner detection algorithm computes the Harris response at each pixel in the image. The Harris response is a measure of the corner ness of a pixel. Pixels with a high Harris response are considered to be corners.

Original Image

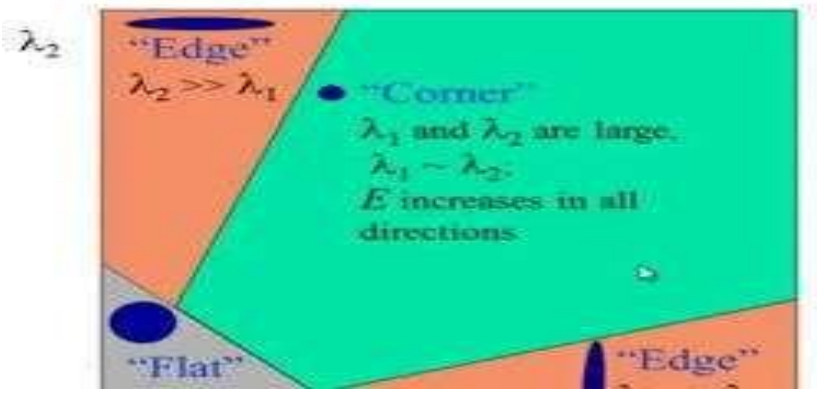
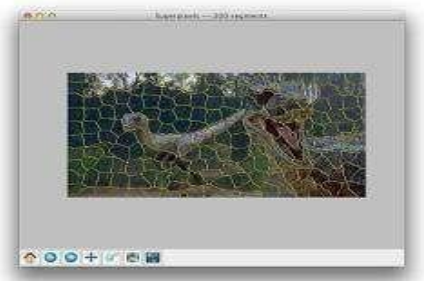


IMAGE AFTER HARRIS CORNER DETECTION

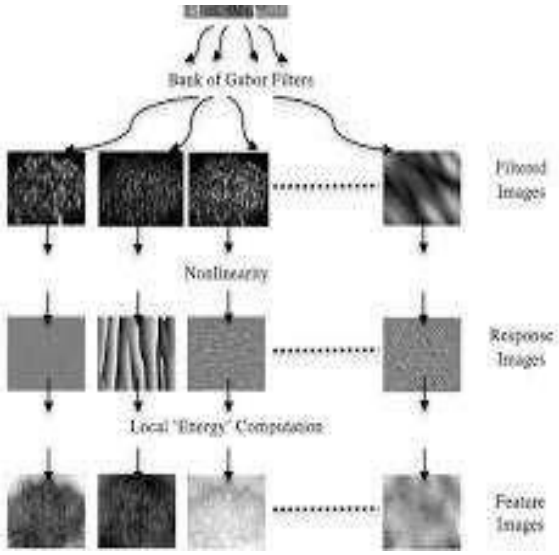
- Region-based feature extraction: The SLIC super pixel algorithm is a popular region-based feature extraction algorithm that divides an image into super pixels. Super pixels are small, homogeneous regions in an image. The SLIC super pixel algorithm uses a k-means clustering algorithm to divide the image into super pixels.



ORIGINAL IMAGE

IMAGE AFTER SLIC SUPERPIXEL SEGMENTATION

- **Texture analysis:** The Gabor filter is a popular texture analysis algorithm that uses a set of filters to extract texture features from an image. The Gabor filter is a bandpass filter that is tuned to a specific frequency and orientation. The Gabor filter is applied to the image, and the output of the filter is used to extract texture features.



ORIGINAL IMAGE

IMAGE AFTER GABOR FILTER TEXTURE ANALYSIS

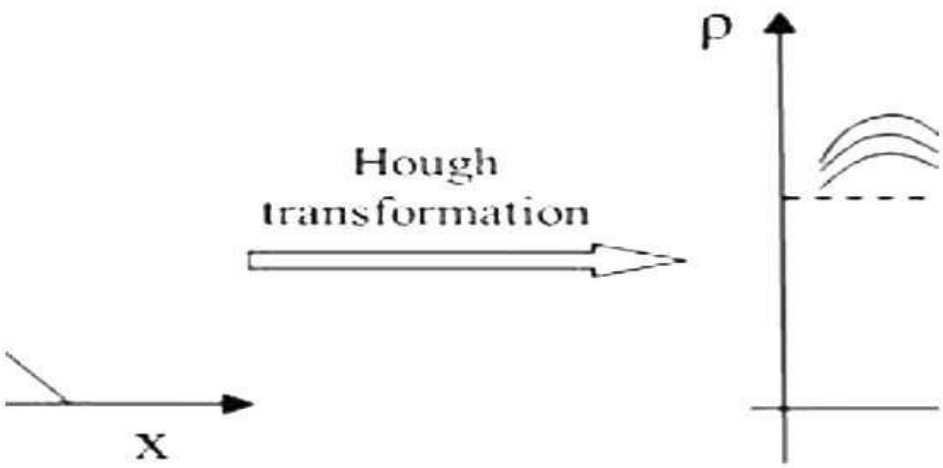
Feature extraction is a fundamental step in many computer vision applications. By carefully selecting and extracting relevant features from images, we can improve the performance of computer vision algorithms and make them more accurate and robust.

2.3 Line Detectors

Line detectors are algorithms that identify and locate lines in images. Lines are important features in images because they often correspond to object boundaries or other important structures.

2.3.1 Hough Transform

The Hough Transform is a popular line detection algorithm that uses a voting scheme to identify lines in an image. The Hough Transform works by transforming each pixel in the image into a set of parameter values that correspond to a line that passes through that pixel. The parameter values are then accumulated in a Hough accumulator, and the pixels with the highest accumulator values are considered to be line pixels.



Hough Transform

Derivation of the Hough Transform

The Hough Transform can be derived from the equation of a line:

$y = mx + b$

where:

y is the y-coordinate of a point on the line

x is the x-coordinate of a point on the line

m is the slope of the line

b is the y-intercept of the line

To transform a pixel into a set of parameter values, we can rewrite the equation of the line as:

$b = -mx + y$

This equation represents a line in parameter space, where the x-axis represents the slope of the line and the y-axis represents the y-intercept of the line.

To apply the Hough Transform, we first compute the gradient magnitude and direction at each pixel in the image. The gradient magnitude represents the strength of the edge at that pixel, and the gradient direction represents the orientation of the edge at that pixel.

Next, we quantize the gradient magnitude and direction into a set of discrete values. This quantization step is necessary because the Hough accumulator is a discrete array.

Finally, we apply the following algorithm to each pixel in the image:

- **For each quantized gradient magnitude and direction:**
- Compute the slope and y-intercept of the line that passes through the pixel and has the given gradient magnitude and direction.
- Increment the Hough accumulator at the corresponding parameter value.
- The pixels with the highest accumulator values are considered to be line pixels.

2.3.2 Corners

Corners are points in an image where two edges intersect. Corners are important features in images because they often correspond to object corners or other important structures.

Harris Corner Detector

The Harris corner detector is a popular corner detection algorithm that uses the second moment matrix to identify corners. The Harris corner detector computes the Harris response at each pixel in the image. The Harris response is a measure of the corner ness of a pixel. Pixels with a high Harris response are considered to be corners.

H = [Ixx Ixy; Ixy Iyy]

where:

- Ixx is the second moment of the image intensity in the x-direction
- Ixy is the second moment of the image intensity in the x-y direction
- Iyy is the second moment of the image intensity in the y-direction

The Harris response is computed as follows:

R = det(H) - tr(H)^2 / 4

where:

- R is the Harris response
- det(H) is the determinant of the second moment matrix
- tr(H)^2 is the square of the trace of the second moment matrix
- Pixels with a Harris response greater than a certain threshold are considered to be corners.

2.3.3 Hessian Affine Corner Detector

The Hessian Affine corner detector is another popular corner detection algorithm that uses the Hessian matrix to identify corners. The Hessian matrix is a second-order derivative matrix that is used to compute the curvature of a surface. The Hessian Affine corner detector computes the eigenvalues of the Hessian matrix at each pixel in the image. Pixels with eigenvalues that are both positive or both negative are considered to be corners.

The Hessian matrix is computed as follows:

H = [Ixx Ixy; Ixy Iyy]

where:

- Ixx is the second derivative of the image intensity in the x-direction
- Ixy is the second derivative of the image intensity in the x-y direction
- Iyy is the second derivative of the image intensity in the y-direction

The eigenvalues of the Hessian matrix are computed as follows:

lambda1, lambda2 = np.linalg.eig(H)

where:

- lambda1 is the first eigenvalue of the Hessian matrix
- lambda2 is the second eigenvalue of the Hessian matrix

Pixels with eigenvalues that are both positive or both negative are considered to be corners.

Line detectors, corners, and the Hough Transform are important tools for feature extraction in computer vision. These algorithms.

2.3.4 Orientation Histogram

An orientation histogram is a feature descriptor used in computer vision to represent the distribution of edge directions in an image region. It is a low-dimensional representation of the local structure of an image and is often used as a feature for object recognition and image matching.

How to compute an Orientation Histogram

The orientation histogram is computed by quantizing the edge orientations in an image region into a set of bins and then counting the number of edge pixels in each bin. The edge orientations are typically computed using a gradient magnitude and direction operator, such as the Sobel or Canny edge detector.

Here's the step-by-step process for computing an orientation histogram:

1. **Apply an edge detector to the image:** Use an edge detection algorithm, such as Canny or Sobel, to identify the edges in the image.
2. **Calculate the gradient magnitude and direction for each edge pixel:** For each edge pixel, compute the gradient magnitude and direction using the edge detector's output.
3. **Quantize the gradient directions into bins:** Divide the gradient directions into a predefined number of bins, typically 8 or 16.
4. **Count the edge pixels in each bin:** For each edge pixel, determine its corresponding bin based on its quantized gradient direction. Increment the count for that bin.
5. **Normalize the histogram (optional):** Divide the number of edge pixels in each bin by the total number of edge pixels in the image. This makes the histogram invariant to the size of the region.

Example of an Orientation Histogram:

Consider an image with several edges oriented in different directions. After applying an edge detector and quantizing the gradient directions into 8 bins, the orientation histogram might look like this:

Bin	Gradient Direction	Count
1	0-22.5 degrees	10
2	22.5-45 degrees	25
3	45-67.5 degrees	30
4	67.5-90 degrees	40
5	90-112.5 degrees	20
6	112.5-135 degrees	15
7	135-157.5 degrees	10
8	157.5-180 degrees	5

This histogram indicates that the image has a strong concentration of edges oriented in the 45-90 degree range.

Applications of Orientation Histograms

Orientation histograms are widely used in computer vision applications, including:

- **Object recognition:** Orientation histograms can be used to extract features from objects and compare them to reference models for object recognition tasks.
- **Image matching:** Orientation histograms can be used to match similar regions in different images, enabling tasks like image stitching and panorama creation.
- **Texture analysis:** Orientation histograms can capture the texture patterns in an image, making them useful for texture classification and identification.
- **Image segmentation:** Orientation histograms can provide clues about image boundaries and help in segmenting an image into distinct regions.

Orientation histograms are a simple yet effective tool for extracting local image features based on edge orientations. Their invariance to illumination and rotational changes makes them a valuable asset in various computer vision applications.

2.3.5 Scale-Invariant Feature Transform (SIFT)

The Scale-Invariant Feature Transform (SIFT) is a feature descriptor used in computer vision to describe local features in images. It is a powerful and versatile feature descriptor that is invariant to scale, rotation, and illumination changes, making it a valuable tool for object recognition, image matching, and other computer vision tasks.

SIFT Algorithm Overview:

The SIFT algorithm consists of four main stages:

1. Scale-space extrema detection:

- The image is convolved with Gaussian kernels at multiple scales to create a scale-space representation.
- Scale-space extrema, or key points, are identified by finding points that are maxima or minima of the Difference of Gaussian (DoG) filter responses across different scales.

2. Key point localization:

- The precise location and scale of each key point are refined by fitting a quadratic function to the DoG filter responses in the vicinity of the key point.
- Sub pixel accuracy is achieved by interpolating the DoG filter responses.

3. Orientation assignment:

- The dominant orientation of each key point is determined by analyzing the gradient magnitudes and directions of the pixels in the neighborhood of the key point.
- The orientation is assigned to the key point and used for subsequent feature description.

4. Feature descriptor generation:

- A descriptor vector is computed for each key point by dividing the key point's neighborhood into a grid of 4x4 subregions.
- Within each subregion, the gradient magnitudes and directions are weighted by a Gaussian window and quantized into orientation histograms.
- The concatenated histograms from all subregions form the SIFT descriptor vector.

Invariance to Scale and Rotation:

The SIFT algorithm achieves invariance to scale by detecting key points at multiple scales and using the scale of the key point to normalize the feature descriptor. This ensures that the descriptor remains consistent across different image scales.

Rotation invariance is achieved by assigning an orientation to each key point based on the local image structure. The feature descriptor is then computed relative to this orientation, ensuring that it remains consistent across different image rotations.

Applications of SIFT

The SIFT feature descriptor has been widely adopted in various computer vision applications due to its robustness and effectiveness. Some common applications include:

- **Object recognition:** SIFT features are often used to represent objects in images and match them to reference models for object recognition tasks.
- **Image matching:** SIFT features are used to find corresponding regions in different images, enabling tasks like image stitching, panorama creation, and image registration.
- **Structure from motion:** SIFT features are used to track feature points across a sequence of images to reconstruct the 3D structure of the scene.
- **Image retrieval:** SIFT features are used to index and retrieve similar images from large image databases.

SIFT is a powerful and versatile feature descriptor that has become a staple in computer vision due to its robustness, invariance properties, and effectiveness in a wide range of

applications. Its ability to handle scale, rotation, and illumination changes makes it a valuable tool for tasks like object recognition, image matching, and structure from motion.

2.3.6 Speeded-Up Robust Features (SURF)

SURF is a feature descriptor used in computer vision to describe local features in images. It is a fast and efficient alternative to SIFT, while still maintaining good performance in terms of scale, rotation, and illumination invariance. SURF is often used for object recognition, image matching, and other computer vision tasks.

SURF Algorithm Overview:

The SURF algorithm consists of four main stages:

1. **Hessian-based interest point detection:**
    - Key points are identified by finding points with high second-order moment matrix (Hessian matrix) determinant.
    - The Hessian matrix captures the local curvature of the image intensity, and high determinant values indicate significant local changes in intensity.
    - Scale selection:
    - The scale of each key point is determined by analyzing the scale at which the Hessian response is strongest.
    - This scale represents the characteristic scale of the local structure around the key point.
3. **Orientation assignment:**
    - The dominant orientation of each key point is determined by analyzing the Haar wavelet responses at the key point and its neighboring pixels.
    - Haar wavelets capture local image patterns, and the dominant orientation corresponds to the direction of the strongest pattern.
4. **Feature descriptor generation:**

- A descriptor vector is computed for each key point by dividing the key point's neighborhood into a circular region and a set of radial and angular subregions.
- Within each subregion, the Haar wavelet responses are weighted by a Gaussian window and used to create a descriptor vector.

Fast and Efficient SURF

SURF achieves its speed and efficiency by utilizing Haar wavelets and integral images. Haar wavelets are computationally efficient to compute, and integral images allow for rapid calculation of Haar wavelet responses. Additionally, the use of a circular region instead of a rectangular grid reduces the number of computations.

Invariance to Scale and Rotation

SURF achieves invariance to scale by selecting the scale at which the Hessian response is strongest and normalizing the feature descriptor accordingly. This ensures that the descriptor remains consistent across different image scales.

Rotation invariance is achieved by assigning an orientation to each keypoint based on the local image structure using Haar wavelets. The feature descriptor is then computed relative to this orientation, ensuring that it remains consistent across different image rotations.

Applications of SURF

SURF has been successfully applied in various computer vision tasks, including:

- **Object recognition:** SURF features are used to represent objects in images and match them to reference models for object recognition tasks.
- **Image matching:** SURF features are used to find corresponding regions in different images, enabling tasks like image stitching, panorama creation, and image registration.
- **3D reconstruction:** SURF features are used to track feature points across a sequence of images to reconstruct the 3D structure of the scene.

- **Image retrieval:** SURF features are used to index and retrieve similar images from large image databases.

SURF is a powerful and efficient feature descriptor that has gained popularity due to its speed, robustness, and effectiveness in a range of computer vision applications. Its ability to handle scale, rotation, and illumination changes while maintaining computational efficiency makes it a valuable tool for tasks like object recognition, image matching, and 3D reconstruction.

2.3.7 Histogram of Oriented Gradients (HOG)

HOG is a feature descriptor used in computer vision to describe local features in images. It is a powerful and versatile feature descriptor that is invariant to illumination and geometric transformations, making it a valuable tool for object detection and image classification.

HOG Algorithm Overview:

The HOG algorithm consists of three main stages:

1. Cell computation:

- The image is divided into a regular grid of cells.
- For each cell, the gradient magnitude and direction are computed for each pixel within the cell.

2. Histogram computation:

- The gradient magnitudes and directions are quantized into a predefined number of bins, typically 8 or 16.
- A histogram of gradient orientations is computed for each cell.

3. Block normalization:

- The histograms from neighboring cells are grouped into blocks.
- The histograms in each block are normalized by dividing them by the square root of the sum of their squares. This normalization helps to reduce the impact of illumination changes.

Invariance to Illumination and Geometric Transformations

HOG achieves invariance to illumination by normalizing the histograms in each block. This normalization ensures that the histograms are not affected by large changes in overall image brightness.

Geometric invariance is achieved by using a dense grid of cells and blocks. This dense representation allows the HOG descriptor to capture local image structure even when the object is subject to geometric transformations, such as scale changes, rotations, and translations.

Applications of HOG:

HOG is widely used in computer vision applications, including:

- **Object detection:** HOG features are often used to represent objects in images and train object detection classifiers.
- **Image classification:** HOG features can be used to classify images into different categories, such as pedestrians, vehicles, and faces.
- **Image segmentation:** HOG features can be used to segment images into distinct regions, such as foreground and background.
- **Action recognition:** HOG features can be used to recognize human actions in videos.

HOG is a powerful and versatile feature descriptor that has become a standard for object detection and image classification. Its invariance to illumination and geometric transformations, along with its computational efficiency, makes it a valuable tool for a wide range of computer vision tasks.

2.3.8 Gradient Location and Orientation Histogram (GLOH)

GLOH is a feature descriptor used in computer vision to describe local features in images. It is a powerful and robust feature descriptor that is invariant to scale, rotation, and illumination changes, making it a valuable tool for object recognition and image matching.

GLOH Algorithm Overview

The GLOH algorithm consists of four main stages:

1. Scale-invariant interest point detection:

- Key points are identified using a scale-invariant interest point detector, such as the Harris corner detector.
- Key points represent locations in the image where there is significant local change in intensity.

2. Laplacian of Gaussian (LoG) filter application:

- The LoG filter is applied to the image at multiple scales to create a scale-space representation.
- The LoG filter emphasizes edges and corners, making it suitable for identifying key points.

3. Gradient magnitude and direction computation:

- The gradient magnitude and direction are computed for each pixel in the scale-space representation around each key point.
- The gradient magnitude indicates the strength of the edge or corner, while the gradient direction indicates its orientation.

4. Feature descriptor generation:

- A descriptor vector is computed for each key point by dividing the key point's neighborhood into concentric rings and radial subregions.
- Within each subregion, the gradient magnitudes and orientations are weighted by a Gaussian window and quantized into orientation histograms.
- A concatenation of the orientation histograms from all subregions forms the GLOH descriptor vector.

GLOH Derivation

The Gradient Location and Orientation Histogram (GLOH) feature descriptor is derived from the local image structure around a key point. The key point is first detected using a

scale-invariant interest point detector, such as the Harris corner detector. Next, the Laplacian of Gaussian (LoG) filter is applied to the image at multiple scales to create a scale-space representation. The LoG filter emphasizes edges and corners, making it suitable for identifying key points.

The gradient magnitude and direction are then computed for each pixel in the scale-space representation around each key point. The gradient magnitude indicates the strength of the edge or corner, while the gradient direction indicates its orientation. The gradient magnitudes are normalized before quantization to make the descriptor invariant to illumination changes.

The GLOH descriptor vector is computed by dividing the key point's neighborhood into concentric rings and radial subregions. Within each subregion, the gradient magnitudes and orientations are weighted by a Gaussian window and quantized into orientation histograms. A concatenation of the orientation histograms from all subregions forms the GLOH descriptor vector.

GLOH Invariance to Scale, Rotation, and Illumination

GLOH achieves invariance to scale by identifying key points at multiple scales and using the scale of the key point to normalize the feature descriptor. This ensures that the descriptor remains consistent across different image scales.

Rotation invariance is achieved by computing the gradient directions relative to the dominant orientation of the key point. This orientation is determined by analyzing the gradient magnitudes and directions in the neighborhood of the key point.

Illumination invariance is achieved by normalizing the gradient magnitudes before quantization. This normalization helps to reduce the impact of changes in overall image brightness.

GLOH Applications

GLOH has been successfully applied in various computer vision tasks, including:

- **Object recognition:** GLOH features are used to represent objects in images and match them to reference models for object recognition tasks.
- **Image matching:** GLOH features are used to find corresponding regions in different images, enabling tasks like image stitching, panorama creation, and image registration.
- **3D reconstruction:** GLOH features are used to track feature points across a sequence of images to reconstruct the 3D structure of the scene.
- **Image retrieval:** GLOH features are used to index and retrieve similar images from large image databases.

GLOH is a powerful and robust feature descriptor that has gained popularity due to its invariance to scale, rotation, and illumination changes. Its ability to handle these variations while maintaining effectiveness makes it a valuable tool for tasks like object recognition, image matching, and 3D reconstruction.

2.4 Scale-Space Analysis

Scale-space analysis is a technique used in computer vision to represent and analyze images at different scales. It is a powerful tool for feature extraction, object recognition, and image matching.

Definition of Scale-Space

A scale-space representation is a collection of images obtained by applying a smoothing filter to an original image at different scales. The smoothing filter removes high-frequency details while preserving low-frequency information. The scale of the image is determined by the size of the smoothing filter.

2.4.1 Scale-Space Properties

A scale-space representation should have the following properties:

- **Scale invariance:** The representation should be invariant to changes in scale. This means that features should be detected and represented consistently across different image scales.

- **Locality:** The representation should be local. This means that features should be represented in a way that preserves their spatial relationships.
- **Edge preservation:** The representation should preserve edges. Edges are important features in images, and they should be represented accurately at all scales.

Scale-Space Construction

There are many different methods for constructing scale-space representations. Some common methods include:

- **Gaussian smoothing:** This method involves convolving the image with a Gaussian kernel at different scales. The Gaussian kernel is a bell-shaped curve that assigns weights to pixels based on their distance from the center of the kernel.
- **Laplacian of Gaussian (LoG) filtering:** This method involves applying the LoG filter to the image at different scales. The LoG filter is a second-order derivative filter that emphasizes edges and corners.
- **Wavelet-based methods:** Wavelet-based methods represent the image as a set of wavelet coefficients at different scales. Wavelets are localized functions that can capture both spatial and frequency information.

Scale-Space Applications

Scale-space analysis has a wide range of applications in computer vision, including:

- **Feature extraction:** Scale-space analysis can be used to extract features from images at different scales. This is important for tasks such as object recognition and image matching.
- **Object recognition:** Scale-space analysis can be used to recognize objects in images by matching features extracted from the image to features in a reference database.

- **Image matching:** Scale-space analysis can be used to match images by finding corresponding features in the two images. This is useful for tasks such as image stitching and panorama creation.
- **Image registration:** Scale-space analysis can be used to register images by aligning features in the two images. This is useful for tasks such as motion tracking and image stabilization.

Examples of Scale-Space Analysis

Here are some examples of how scale-space analysis is used in computer vision:

- **SIFT (Scale-Invariant Feature Transform):** SIFT is a popular feature descriptor that uses scale-space analysis to detect and describe features in images.
- **SURF (Speeded-Up Robust Features):** SURF is another popular feature descriptor that uses scale-space analysis to detect and describe features in images. SURF is faster than SIFT, but it is not as accurate.
- **DOG (Difference of Gaussians):** DOG is a method for detecting key points in images. DOG uses two Gaussian kernels at different scales to detect locations where the intensity of the image changes significantly.

Scale-space analysis is a powerful tool for feature extraction, object recognition, and image matching. It is a versatile technique that can be used to solve a wide range of problems in computer vision.

2.5 Image Pyramids

An image pyramid is a multi-scale representation of an image. It is created by repeatedly applying a smoothing filter to an image and then subsampling the smoothed image, usually by a factor of 2 along each coordinate direction. The resulting image is then subjected to the same procedure, and the cycle is repeated multiple times. Each cycle of this process results in a smaller image with increased smoothing, but with decreased spatial sampling density (that is, decreased image resolution). If illustrated graphically,

the entire multi-scale representation will look like a pyramid, with the original image on the bottom and each cycle's resulting smaller image stacked one atop the other.

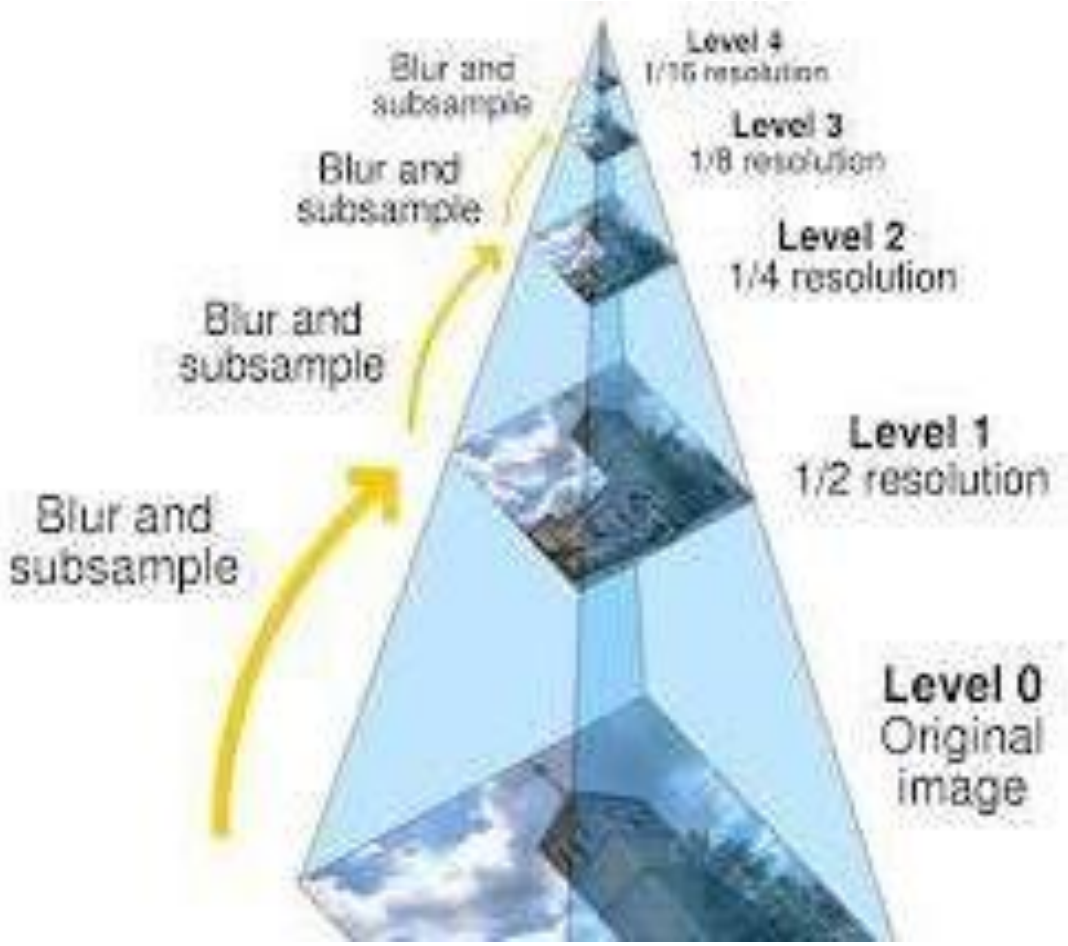


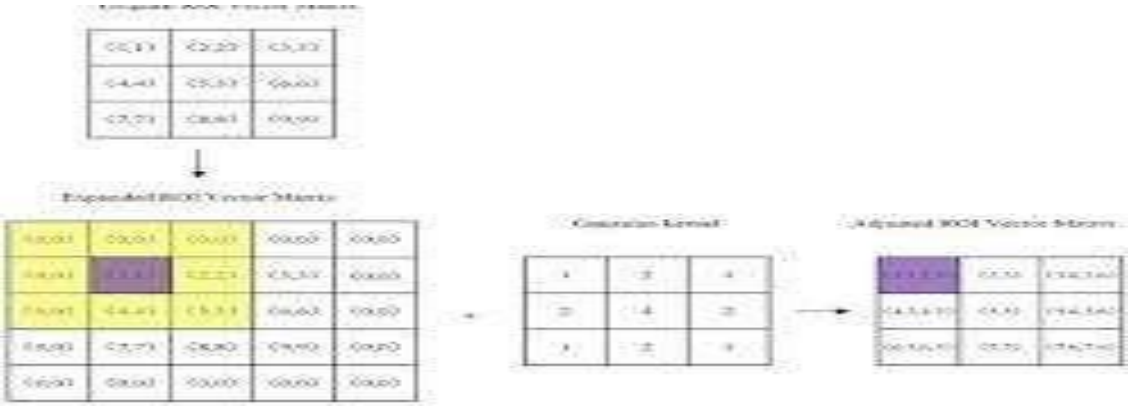
Image pyramids are used in a variety of computer vision tasks, including:

- Image processing: Image pyramids can be used to reduce noise, sharpen images, and improve edge detection.
- Object recognition: Image pyramids can be used to represent objects at different scales, which can be helpful for object recognition tasks.
- Image matching: Image pyramids can be used to find corresponding regions in different images, which is useful for tasks such as image stitching and panorama creation.

2.5.1 Gaussian Derivative Filters

Gaussian derivative filters are a type of derivative filter that uses a Gaussian kernel. The Gaussian kernel is a bell-shaped curve that assigns weights to pixels based on their distance from the center of the kernel. The weights decrease as the distance from the center increases, which means that the filter is more sensitive to pixels that are closer to the center.

Gaussian derivative filters are often used in computer vision to detect edges and corners. The first derivative of the Gaussian kernel is the Gaussian gradient filter, which is used to detect edges. The second derivative of the Gaussian kernel is the Laplacian of Gaussian (LoG) filter, which is used to detect corners.



Gaussian derivative filters have several advantages over other types of derivative filters:

- They are less sensitive to noise.
- They are more likely to detect true edges and corners.
- They are more invariant to scale.
- Applications of Image Pyramids and Gaussian Derivative Filters
- Image pyramids and Gaussian derivative filters are used in a variety of computer vision applications, including:
- Edge detection: Gaussian derivative filters can be used to detect edges in images.
- Corner detection: Gaussian derivative filters can be used to detect corners in images.

- Scale-invariant feature detection: Image pyramids can be used to detect features in images at different scales.
- Image registration: Image pyramids can be used to register images with different sizes or scales.
- Image fusion: Image pyramids can be used to fuse images from different sources.

Image pyramids and Gaussian derivative filters are powerful tools for image processing and computer vision. They are used in a wide variety of applications and have been shown to be very effective.

2.5.2 Gabor Filters

Gabor filters are a type of linear filter that are used in a variety of image processing and computer vision tasks. They are named after the Hungarian physicist Dennis Gabor, who first introduced them in 1946. Gabor filters are often used to extract features from images, such as edges, corners, and textures. They are also used in tasks such as image denoising, image recognition, and image synthesis.

Derivation of Gabor Filters

A Gabor filter is a complex-valued function that is defined as the product of a Gaussian function and a complex exponential function. The Gaussian function is used to define the spatial support of the filter, while the complex exponential function is used to define the frequency and orientation of the filter.The derivation of a Gabor filter can be written as:

$G(x, y) = e^{(-\pi f^{(2)}(x^2 + y^2))} * e^{(2\pi i(fx + fy))}$

where:

- G(x, y) is the Gabor filter
- f is the frequency of the filter
- θ is the orientation of the filter
- x and y are the spatial coordinates

Properties of Gabor Filters

Gabor filters have several properties that make them useful for image processing and computer vision tasks. These properties include:

- **Localization:** Gabor filters are localized in both spatial and frequency domains. This means that they can extract features from images that are both spatially localized and have a specific frequency content.
- **Scale invariance:** Gabor filters can be scaled to different sizes without affecting their shape. This makes them useful for extracting features from images at different scales.
- **Orientation invariance:** Gabor filters can be rotated to different orientations without affecting their shape. This makes them useful for extracting features from images that have different orientations.

Applications of Gabor Filters

Gabor filters are used in a variety of image processing and computer vision tasks. Some of these applications include:

- **Edge detection:** Gabor filters can be used to detect edges in images. The edges are detected by the filter's response to changes in intensity.
- **Corner detection:** Gabor filters can be used to detect corners in images. The corners are detected by the filter's response to changes in both intensity and orientation.
- **Texture analysis:** Gabor filters can be used to analyze the texture of images. The texture is analyzed by the filter's response to the spatial distribution of intensity.
- **Image recognition:** Gabor filters can be used to recognize objects in images. The objects are recognized by matching the features extracted from the image with the features of known objects.
- **Image synthesis:** Gabor filters can be used to synthesize images. The images are synthesized by combining the features extracted from different images.

Gabor filters are a powerful tool for image processing and computer vision. They are used in a wide variety of applications and have been shown to be very effective.

2.6 Introduction to Discrete Wavelet Transform (DWT)

Discrete wavelet transform (DWT) is a mathematical tool used to decompose a signal into a series of wavelets. Wavelets are localized functions that can capture both spatial and frequency information. DWT is widely used in image processing and computer vision for tasks such as image compression, feature extraction, and image denoising.

DWT Algorithm:

The DWT algorithm consists of two main stages:

1. **Decomposition:** The image is decomposed into a series of lower-resolution images, called approximation coefficients (A), and higher-resolution images, called detail coefficients (D). This process is repeated multiple times, resulting in a multi-resolution representation of the image.
2. **Reconstruction:** The image can be reconstructed from the approximation and detail coefficients. This process is the inverse of decomposition.

DWT Properties:

DWT has several properties that make it useful for image processing and computer vision tasks:

- **Multi-resolution analysis:** DWT provides a multi-resolution representation of an image, which allows for analysis of the image at different scales.
- **Energy compaction:** DWT concentrates the energy of an image in the approximation coefficients, while the detail coefficients contain mostly noise. This property is useful for image compression.
- **Feature extraction:** DWT can extract features from images, such as edges, corners, and textures. These features can be used for tasks such as image recognition and object detection.

Applications of DWT in Computer Vision

DWT is used in a variety of computer vision tasks, including:

- **Image compression:** DWT is a popular method for image compression. The approximation coefficients are quantized and stored, while the detail coefficients are discarded or quantized more coarsely.
- **Feature extraction:** DWT can be used to extract features from images, such as edges, corners, and textures. These features can be used for tasks such as image recognition and object detection.
- **Image denoising:** DWT can be used to remove noise from images. The noise is typically concentrated in the detail coefficients, which can be discarded or thresholder.
- **Image registration:** DWT can be used to register images with different sizes or scales. The approximation coefficients are used to match the images, while the detail coefficients are used to refine the alignment.

DWT is a powerful tool for image processing and computer vision. It is used in a wide variety of applications and has been shown to be very effective.

2.7 Image segmentation

Image Segmentation: In computer vision, image segmentation is the process of partitioning an image into multiple segments. The goal of segmenting an image is to change the representation of an image into something that is more meaningful and easier to analyze. It is usually used for locating objects and creating boundaries.

It is not a great idea to process an entire image because many parts in an image may not contain any useful information. Therefore, by segmenting the image, we can make use of only the important segments for processing.

An image is basically a set of given pixels. In image segmentation, pixels which have similar attributes are grouped together. Image segmentation creates a pixel-wise mask

for objects in an image which gives us a more comprehensive and granular understanding of the object.

Uses:

1. Used in self-driving cars. Autonomous driving is not possible without object detection which involves segmentation.
2. Used in the healthcare industry. Helpful in segmenting cancer cells and tumours using which their severity can be gauged.

There are many more uses of image segmentation.

In this article, we will perform segmentation on an image of the monarch butterfly using a clustering method called K Means Clustering.

K Means Clustering Algorithm:

K Means is a clustering algorithm. Clustering algorithms are unsupervised algorithms which means that there is no labelled data available. It is used to identify different classes or clusters in the given data based on how similar the data is. Data points in the same group are more similar to other data points in that same group than those in other groups.

K-means clustering is one of the most commonly used clustering algorithms. Here, **k** represents the number of clusters.

Let's see how does K-means clustering work –

1. Choose the number of clusters you want to find which is k.
2. Randomly assign the data points to any of the k clusters.
3. Then calculate the center of the clusters.
4. Calculate the distance of the data points from the centers of each of the clusters.
5. Depending on the distance of each data point from the cluster, reassign the data points to the nearest clusters.
6. Again calculate the new cluster center.

7. Repeat steps 4,5 and 6 till data points don't change the clusters, or till we reach the assigned number of iterations.

Requirements:

Make sure you have Python, Numpy, Matplotlib and OpenCV installed.

Code: Read in the image and convert it to an RGB image.

```
import numpy as np
import matplotlib.pyplot as plt
import cv2
 %matplotlib inline
# Read in the image
image = cv2.imread('images/monarch.jpg')
 # Change color to RGB (from BGR)
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
 plt.imshow(image)
```

Now we have to prepare the data for K means. The image is a 3-dimensional shape but to apply k-means clustering on it we need to reshape it to a 2-dimensional array.

Code: Now we will implement the K means algorithm for segmenting an image.

```
# Reshaping the image into a 2D array of pixels and 3 color values (RGB)
```

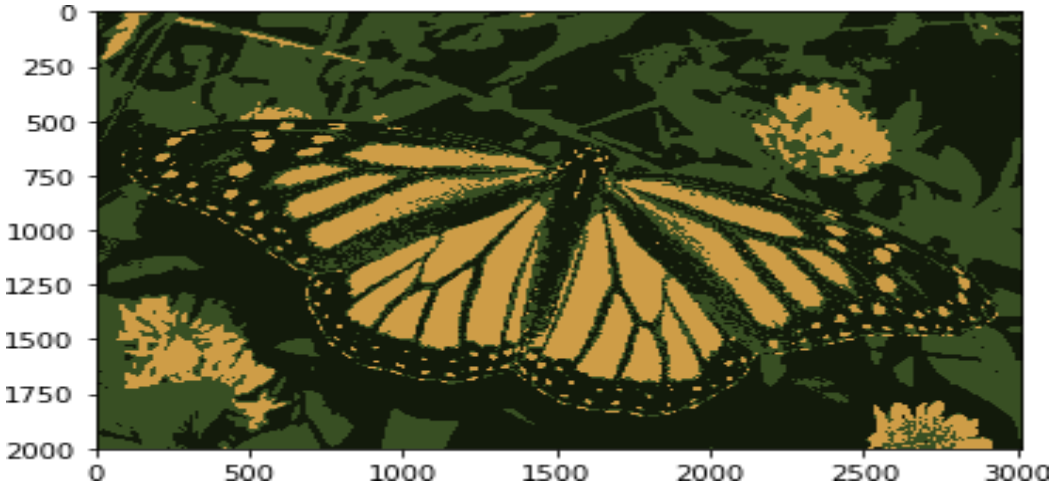
```
pixel_vals = image.reshape((-1,3))
 # Convert to float type
pixel_vals = np.float32(pixel_vals)
```

Taking k = 3, which means that the algorithm will identify 3 clusters in the image.

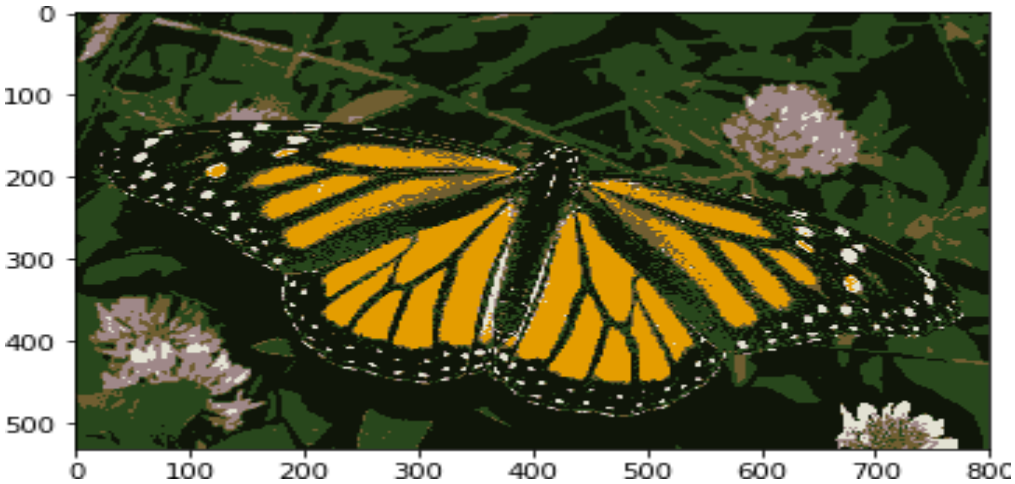
Code:

```
#the below line of code defines the criteria for the algorithm to stop running,
#which will happen is 100 iterations are run or the epsilon (which is the required accuracy)
#becomes 85%
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 100, 0.85)
 # then perform k-means clustering with number of clusters defined as 3
#also random centres are initially choosed for k-means clustering k = 3
retval, labels, centers = cv2.kmeans(pixel_vals, k, None, criteria, 10, cv2.KMEANS_RANDOM_CENTERS)
 # convert data into 8-bit values
centers = np.uint8(centers)
segmented_data = centers[labels.flatten()]
 # reshape data into the original image dimensions
segmented_image = segmented_data.reshape((image.shape))
plt.imshow(segmented_image)
```

Output:



Now if we change the value of **k** to **6**, we get the following **Output:**



As you can see with an increase in the value of ***k***, the image becomes clearer and distinct because the K-means algorithm can classify more classes/cluster of colors. K-means clustering works well when we have a small dataset. It can segment objects in images and also give better results. But when it is applied on large datasets (more number of images), it looks at all the samples in one iteration which leads to a lot of time being taken up.

2.8 SEGMENTATION

Segmentation is the separation of one or more regions or objects in an image based on a discontinuity or a similarity criterion. A region in an image can be defined by its border (edge) or its interior, and the two representations are equal. There are prominently three methods of performing segmentation:

- Pixel Based Segmentation
- Region-Based Segmentation
- Edges based segmentation

2.8.1 Edges based segmentation

Edge-based segmentation contains 2 steps:

Edge Detection: In edge detection, we need to find the pixels that are edge pixels of an object. There are many object detection methods such as Sobel operator, Laplace operator, Canny, etc.

1	**0**	**-1**
2	**0**	**-2**
1	**0**	**-1**

Sobel vertical Operator

+1	**2**	**1**
0	**0**	**0**
-1	**-2**	**-1**

Sobel Horizontal Operator

0	**-1**	**0**
-1	**4**	**-1**
0	**-1**	**0**
Negative Laplace Operator		

Edge Linking: In this step, we try to refine the edge detection by linking the adjacent edges and combine to form the whole object. The edge linking can be performed using any of the two methods below:

- Local Processing: In this method, we used gradient and direction to link the neighborhood edges. If two edges have a similar direction vector then they can be linked.
- Global processing: This method can be done using HOG transformation

Pros :

- This approach is similar to how the humans brain approaches the segmentation task.
- Works well in images with good contrast between object and background.

Limitations:

- Does not work well on images with smooth transitions and low contrast.
- Sensitive to noise.
- Robust edge linking is not trivial and easy to perform.

2.8.2 Region-Based Segmentation

In this segmentation, we grow regions by recursively including the neighboring pixels that are similar and connected to the seed pixel. We use similarity measures such as differences in gray levels for regions with homogeneous gray levels. We use connectivity to prevent connecting different parts of the image.

There are two variants of region-based segmentation:

Top-down approach

- First, we need to define the predefined seed pixel. Either we can define all pixels as seed pixels or randomly chosen pixels. Grow regions until all pixels in the image belongs to the region.

Bottom-Up approach

- Select seed only from objects of interest. Grow regions only if the similarity criterion is fulfilled.

Similarity Measures:

- Similarity measures can be of different types: For the grayscale image the similarity measure can be the different textures and other spatial properties, intensity difference within a region or the distance b/w mean value of the region.

Region merging techniques:

- In the region merging technique, we try to combine the regions that contain the single object and separate it from the background. There are many regions merging techniques such as Watershed algorithm, Split and merge algorithm, etc**.**

Pros:

- Since it performs simple threshold calculation, it is faster to perform.
- Region-based segmentation works better when the object and background have high contrast.

Limitations:

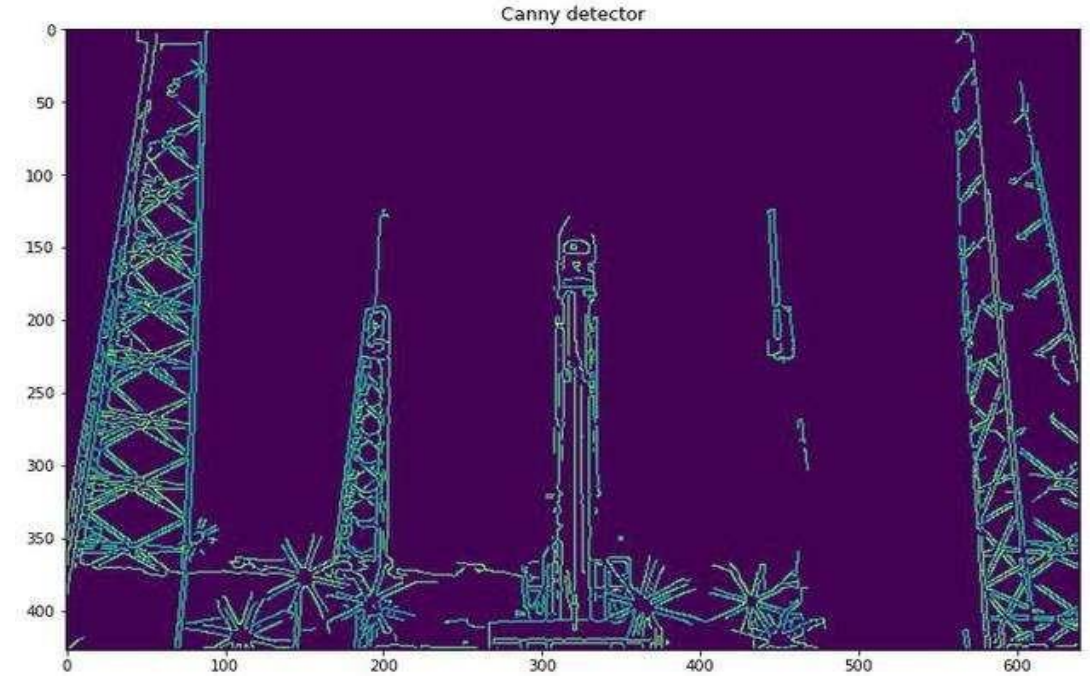
- It did not produce many accurate segmentation results when there are no significant differences b/w pixel values of the object and the background.

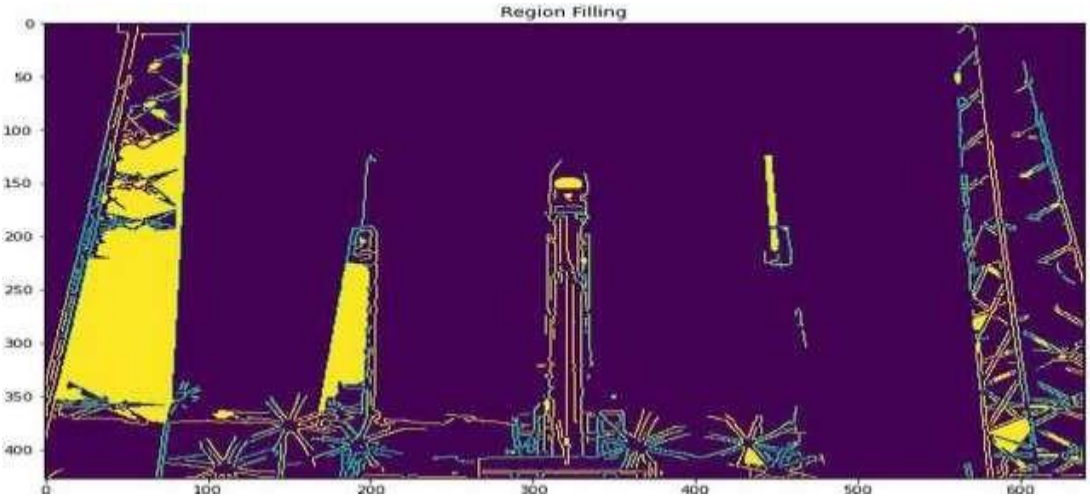
Implementation:

In this implementation, we will be performing edge and region-based segmentation. We will be using scikit image module for that and an image from its dataset provided.

```
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import canny
from skimage import data,morphology
from skimage.color import rgb2gray
import scipy.ndimage as nd
plt.rcParams["figure.figsize"] = (12,8)
%matplotlib inline
 # load images and convert grayscale
rocket = data.rocket()
rocket_wh = rgb2gray(rocket)
 # apply edge segmentation
# plot canny edge detection
edges = canny(rocket_wh)
plt.imshow(edges, interpolation='gaussian')
plt.title('Canny detector')
 # fill regions to perform edge segmentation
fill_im = nd.binary_fill_holes(edges)
plt.imshow(fill_im)
plt.title('Region Filling')
# Region Segmentation
# First we print the elevation map
elevation_map = sobel(rocket_wh)
plt.imshow(elevation_map)
 # Since, the contrast difference is not much. Anyways we will perform it
markers = np.zeros_like(rocket_wh)
markers[rocket_wh < 0.1171875] = 1 # 30/255
```

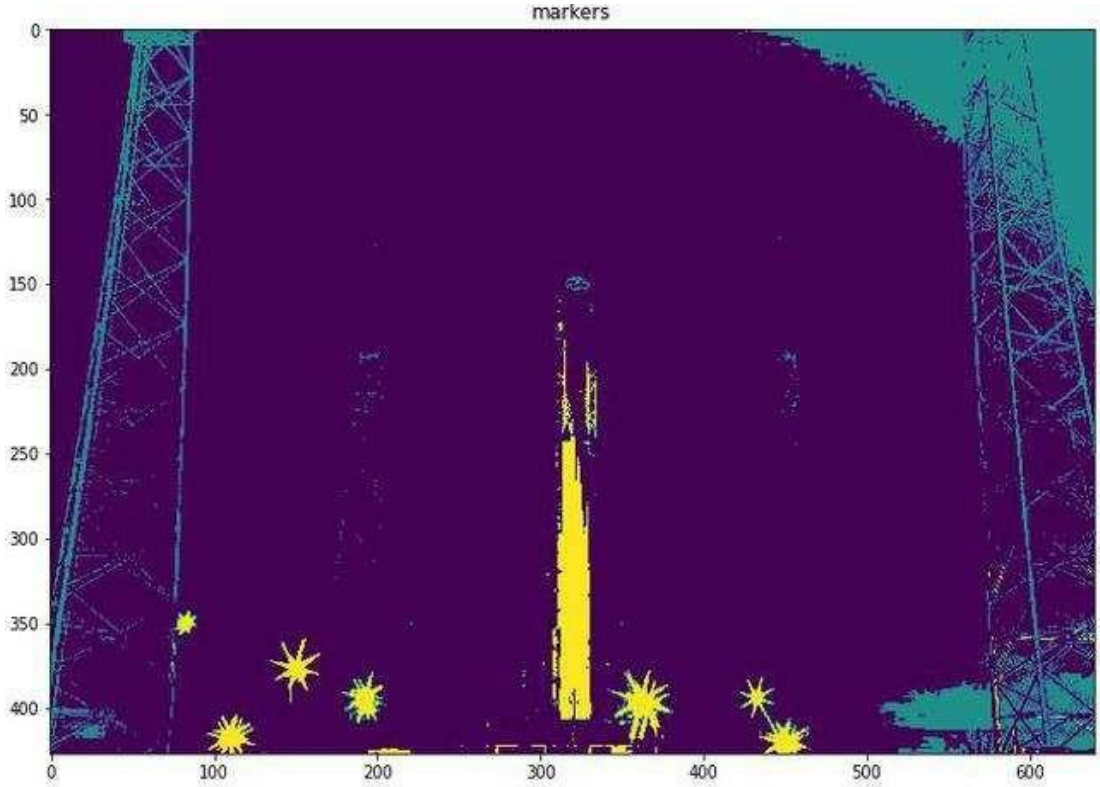
```
markers[rocket_wh > 0.5859375] = 2 # 150/255
plt.imshow(markers)
plt.title('markers')
 # Perform watershed region segmentation
segmentation = morphology.watershed(elevation_map, markers)
 plt.imshow(segmentation)
plt.title('Watershed segmentation')
 # plot overlays and contour
segmentation = nd.binary_fill_holes(segmentation - 1)
label_rock, _ = nd.label(segmentation)
# overlay image with different labels
image_label_overlay = label2rgb(label_rock, image=rocket_wh)
 # code
 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(24, 16), sharey=True)
ax1.imshow(rocket_wh)
ax1.contour(segmentation, [0.8], linewidths=1.8, colors='w')
ax2.imshow(image_label_overlay)
fig.subplots_adjust(**margins)
```

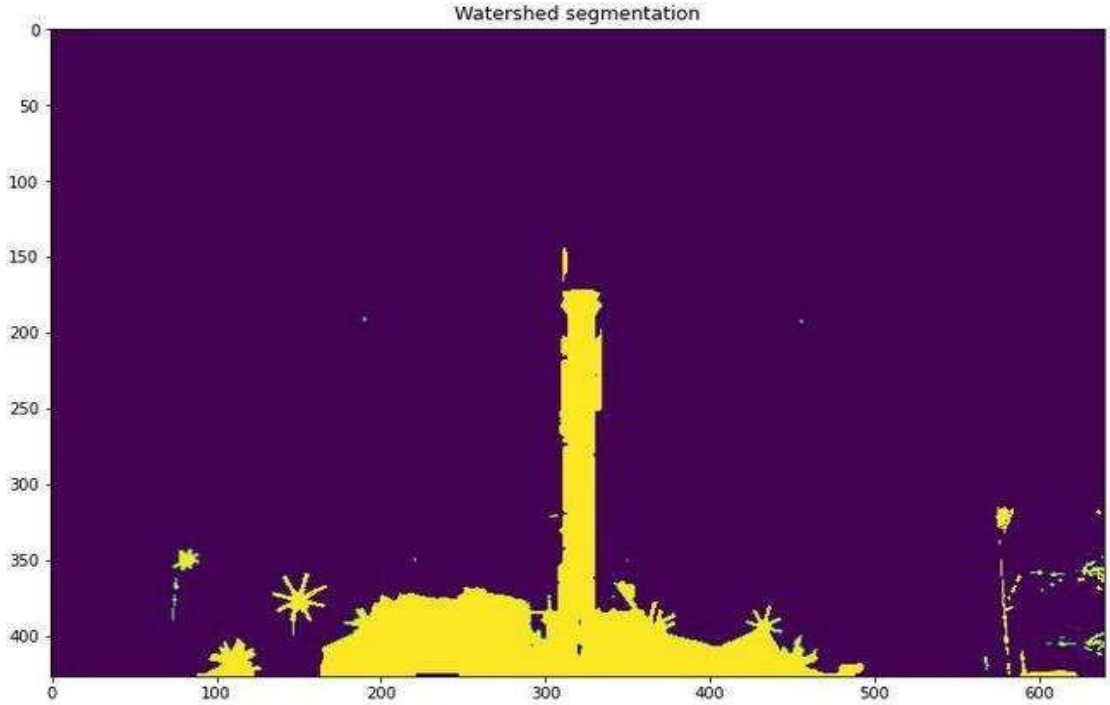


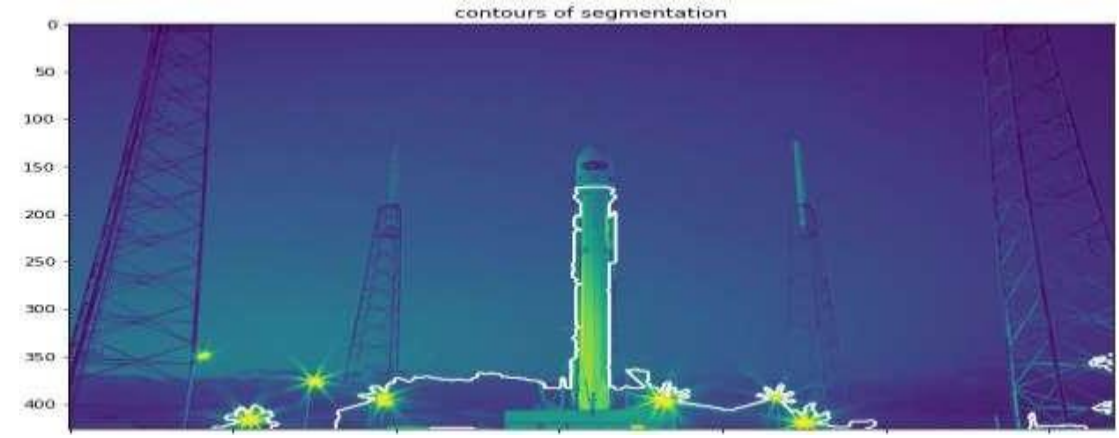


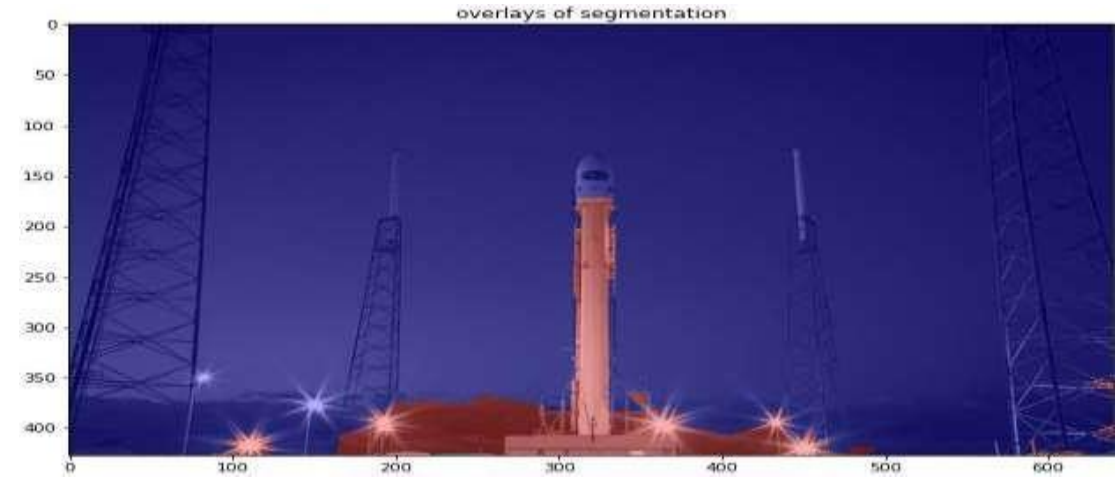
Elevation maps

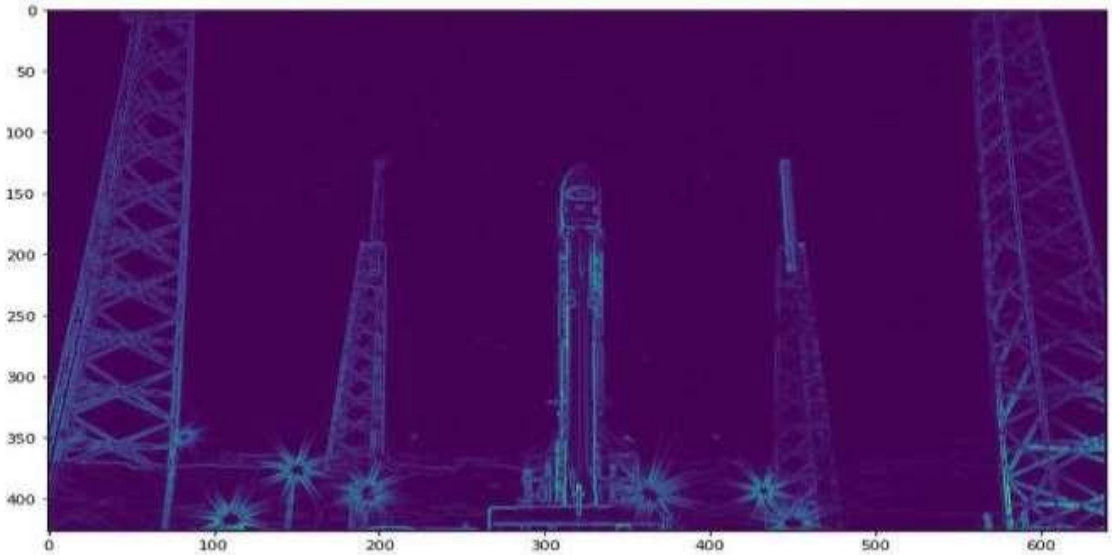
2.38 FEATURE EXTRACTION AND FEATURE SEGMENTATION






contours of segmentation
0
50
100
150
200
250
300
350
400


overlays of segmentation
0
50
100
150
200
250
300
350
400
0
100
200
300
400
500
600


0
50
100
150
200
250
300
350
400
0
100
200
300
400
500
600

2.9 Edge-Based Segmentation Definition and Goals

Edge-based segmentation is a fundamental technique in computer vision that aims to partition an image into distinct regions based on the presence and location of edges. Edges correspond to sharp transitions in intensity or color values, marking the boundaries between different objects or regions in an image. The goal of edge-based segmentation is to detect and extract these edges, enabling the identification and separation of distinct image regions.

Importance of Edge-Based Segmentation

Edge-based segmentation plays a crucial role in various computer vision tasks, including:

1. **Object Recognition:** Identifying and classifying objects in images by recognizing their shapes and boundaries, which are often defined by edges.
2. **Image Analysis and Understanding:** Gaining insights into the structure and content of images by analyzing the distribution of edges, which can reveal the presence of objects, textures, or patterns.
3. **Image Registration:** Aligning images with different sizes or scales by matching corresponding edges, enabling the creation of panoramas or mosaics.
4. **Content-Based Image Retrieval:** Retrieving images from large databases based on their edge content, such as finding images with similar shapes or objects.
5. **Image Editing and Manipulation:** Selectively modifying or removing edges in images, such as enhancing object boundaries or removing unwanted noise or artifacts.

Common Edge Detection Techniques

Several edge detection techniques have been developed to identify and extract edges from images. Some common methods include:

1. **Thresholding:** This method involves applying a threshold to the intensity or color values of pixels, classifying pixels above the threshold as edge pixels and those below as non-edge pixels.

2. **Gradient-based Methods:** These methods measure the magnitude of the gradient, which represents the direction and strength of intensity changes, to identify edge pixels. Popular gradient-based methods include Sobel, Roberts, and Canny edge detectors.
3. **Laplacian of Gaussian (LoG) Filter:** This method applies the LoG filter to the image, which accentuates edges and corners while suppressing noise. Edge pixels are identified as locations where the filtered image has a zero crossing.
4. **Region-Based Edge Detectors:** These methods analyze the local neighborhood of pixels to determine whether they belong to an edge or not. Popular region-based edge detectors include hysteresis and fuzzy logic-based methods.

Advantages and Disadvantages of Edge-Based Segmentation

Edge-based segmentation offers several advantages:

- **Simplicity:** Edge detection algorithms are relatively straightforward to implement and computationally efficient.
- **Effectiveness:** Edge detection can effectively capture the boundaries and shapes of objects in images.
- **Noise Resilience:** Edge detectors are often robust to noise, making them suitable for real-world images.

However, edge-based segmentation also has some limitations:

- **Sensitivity to Noise:** Edge detectors can be sensitive to noise, especially at low intensities or in complex textures.
- **Edge Misdetection:** Edge detectors may miss or misdetect edges due to variations in illumination, contrast, or image quality.
- **Sensitivity to False Edges:** Edge detectors may produce false edges due to noise, texture patterns, or repetitive patterns.

Applications of Edge-Based Segmentation in Computer Vision

Edge-based segmentation has found diverse applications in various computer vision tasks:

1. **Object Detection:** Identifying and locating objects in images by detecting their boundaries and shapes, such as recognizing faces, pedestrians, or vehicles.
2. **Medical Image Analysis:** Segmenting organs, tissues, or lesions in medical images based on their edge patterns, aiding in disease diagnosis and treatment planning.
3. **Image Stitching and Mosaics:** Creating panoramic images or mosaics by stitching together multiple images, aligning them based on detected edges.
4. **Image Enhancement:** Enhancing the visual appearance of images by sharpening edges, removing noise, or highlighting specific features.
5. **Image-Based Recognition:** Recognizing objects, scenes, or actions in images by analyzing the distribution and patterns of edges.

2.9.1 Graph Cuts for Image Segmentation

Graph cuts are a powerful optimization technique used in computer vision for image segmentation. They provide a versatile and efficient framework for partitioning an image into distinct regions based on certain criteria, such as intensity, color, or texture.

Graph Cut Representation

Image segmentation using graph cuts involves constructing a graph representation of the image. The nodes in the graph represent the pixels of the image, and the edges represent the relationships between pixels. Each edge is assigned a weight that reflects the similarity or dissimilarity between the connected pixels.

Energy Minimization

The goal of graph-cut segmentation is to find a partition of the graph into two sets of nodes, typically representing foreground and background, that minimizes a certain energy

function. This energy function typically incorporates factors such as the smoothness of the segmentation and the fidelity to the image data.

Types of Graph Cuts

There are two main types of graph cuts used for image segmentation:

1. Max-flow/Min-cut:

This approach involves maximizing the flow over a cut in the graph. The maximum flow represents the total similarity between the foreground and background regions.

2. Energy minimization:

This approach directly minimizes the energy function over all possible cuts in the graph. The energy function captures both smoothness and data fidelity.

Advantages of Graph Cuts

Graph cuts offer several advantages for image segmentation:

1. Global optimization:

- Graph cuts provide a global optimization framework, ensuring that the segmentation is consistent across the entire image.

2. Smoothness constraint:

- Graph cuts can incorporate smoothness constraints, leading to segmentations with smooth boundaries.

3. Data fidelity:

- Graph cuts can balance smoothness with data fidelity, ensuring that the segmentation adheres to the image data.

4. Efficiency:

Graph-cut algorithms can be implemented efficiently, making them suitable for real-time applications.

Applications of Graph Cuts in Computer Vision

Graph cuts have found wide-ranging applications in various computer vision tasks:

1. Image Segmentation:

Graph cuts are a popular choice for image segmentation due to their ability to handle complex image structures and produce high-quality segmentations.

2. Object Detection:

Graph cuts can be used to identify and localize objects in images by segmenting them from the background and refining their boundaries.

3. Stereo Matching:

Graph cuts can be used to find corresponding points in stereo images, enabling depth estimation and 3D reconstruction.

4. Image Matting:

Graph cuts can be used to separate the foreground object from the background in images, enabling background substitution and image editing.

5. Image Restoration:

Graph cuts can be used to restore images by removing noise or inpainting missing regions, guided by image statistics and spatial coherence.

Graph cuts have emerged as a powerful and versatile technique for image segmentation and other computer vision tasks. Their ability to provide global optimization, enforce smoothness constraints, and balance data fidelity makes them well-suited for a wide range of applications. As computer vision algorithms continue to evolve, graph cuts are likely to remain a valuable tool for image analysis and understanding.

2.9.2 Mean-Shift for Image Segmentation

Mean-Shift is a non-parametric, density-based clustering algorithm that is widely used in image segmentation and computer vision. It is a robust and versatile algorithm that can effectively handle images with complex structures and varying illumination conditions.

Mean-Shift Algorithm Overview:

The Mean-Shift algorithm operates by iteratively shifting data points towards the mode (the highest density) of the distribution of points within a certain radius. This process

continues until the data points converge to local maxima of the density function, representing the clusters in the data.

Mean-Shift Algorithm Steps

The Mean-Shift algorithm for image segmentation can be summarized as follows:

1. Initialization: Start with an initial set of cluster centers, typically represented by pixels in the image.
2. Mode Seeking Iteration: For each pixel in the image, compute the mean shift vector by averaging the locations of its neighbors within a specified radius.
3. Data Point Update: Update the location of the pixel by moving it in the direction of the mean shift vector.
4. Convergence Check: Repeat steps 2 and 3 until the mean shift vectors become negligible or a maximum number of iterations is reached.
5. Clustering Assignment: Assign each pixel to the cluster corresponding to the nearest cluster center.

Mean-Shift Properties

Mean-Shift offers several advantages for image segmentation:

1. Non-parametric:

Mean-Shift does not require prior knowledge of the number of clusters, making it suitable for data with varying cluster structures.

2. Density-based:

Mean-Shift clusters data points based on their density, effectively capturing clusters with arbitrary shapes and sizes.

3. Robust to Outliers:

Mean-Shift is relatively insensitive to outliers, as it is guided by the local density distribution.

4. Adaptive to Illumination:

Mean-Shift can handle images with varying illumination conditions, as it relies on relative intensity differences rather than absolute values.

Applications of Mean-Shift in Computer Vision

Mean-Shift has found diverse applications in various computer vision tasks:

1. Image Segmentation:

Mean-Shift is a popular choice for image segmentation, particularly for images with complex objects and varying illumination.

2. Object Tracking:

Mean-Shift can track objects in videos by following their motion and maintaining consistent clusters over time.

3. Content-Based Image Retrieval:

Mean-Shift can be used to group images with similar features, enabling content-based image retrieval in large databases.

4. Image Analysis and Understanding:

Mean-Shift can aid in image analysis by identifying and characterizing distinct regions in images, providing insights into their content and structure.

Mean-Shift has established itself as a powerful and versatile tool for image segmentation and computer vision. Its non-parametric nature, density-based clustering approach, and robustness to noise and illumination variations make it well-suited for a wide range of applications. As computer vision algorithms continue to evolve, Mean-Shift is likely to remain a valuable tool for image analysis and understanding.

Sure, here are some notes on Mean-Shift for computer vision with images:

2.9.3 Mean-Shift for Image Segmentation

Mean-Shift is a non-parametric, density-based clustering algorithm that is widely used in image segmentation and computer vision. It is a robust and versatile algorithm that can effectively handle images with complex structures and varying illumination conditions.

Mean-Shift Algorithm Overview

The Mean-Shift algorithm operates by iteratively shifting data points towards the mode (the highest density) of the distribution of points within a certain radius. This process continues until the data points converge to local maxima of the density function, representing the clusters in the data.

Mean-Shift Algorithm Steps

The Mean-Shift algorithm for image segmentation can be summarized as follows:

1. Initialization: Start with an initial set of cluster centers, typically represented by pixels in the image.
2. Mode Seeking Iteration: For each pixel in the image, compute the mean shift vector by averaging the locations of its neighbors within a specified radius.
3. Data Point Update: Update the location of the pixel by moving it in the direction of the mean shift vector.
4. Convergence Check: Repeat steps 2 and 3 until the mean shift vectors become negligible or a maximum number of iterations is reached.

Clustering Assignment: Assign each pixel to the cluster corresponding to the nearest cluster center.

Mean-Shift Properties

Mean-Shift offers several advantages for image segmentation:

1. Non-parametric:

Mean-Shift does not require prior knowledge of the number of clusters, making it suitable for data with varying cluster structures.

2. Density-based:

Mean-Shift clusters data points based on their density, effectively capturing clusters with arbitrary shapes and sizes.

3. Robust to Outliers:

Mean-Shift is relatively insensitive to outliers, as it is guided by the local density distribution.

4. Adaptive to Illumination:

Mean-Shift can handle images with varying illumination conditions, as it relies on relative intensity differences rather than absolute values.

Applications of Mean-Shift in Computer Vision

Mean-Shift has found diverse applications in various computer vision tasks:

1. Image Segmentation:

Mean-Shift is a popular choice for image segmentation, particularly for images with complex objects and varying illumination.

2. Object Tracking:

Mean-Shift can track objects in videos by following their motion and maintaining consistent clusters over time.

3. Content-Based Image Retrieval:

Mean-Shift can be used to group images with similar features, enabling content-based image retrieval in large databases.

4. Image Analysis and Understanding:

Mean-Shift can aid in image analysis by identifying and characterizing distinct regions in images, providing insights into their content and structure.

Mean-Shift has established itself as a powerful and versatile tool for image segmentation and computer vision. Its non-parametric nature, density-based clustering approach, and robustness to noise and illumination variations make it well-suited for a wide range of applications. As computer vision algorithms continue to evolve, Mean-Shift is likely to remain a valuable tool for image analysis and understanding.

2.10 Markov Random Fields (MRFs) for Computer Vision

Markov random fields (MRFs) are a powerful statistical modeling framework widely used in computer vision for various tasks, including image segmentation, image restoration, and image registration. MRFs provide a probabilistic representation of the relationships between pixels in an image, enabling the capture of spatial context and dependencies within the image data.

MRF Definition and Properties:

An MRF is an undirected graphical model that represents a set of random variables and the conditional dependencies between them. In the context of image segmentation, the random variables represent the pixels in the image, and the conditional dependencies capture the relationships between neighboring pixels. MRFs possess several key properties that make them well-suited for image analysis:

1. Locality:

The conditional dependencies within an MRF are local, meaning that the value of a pixel depends only on its immediate neighbors.

2. Global Consistency:

MRFs can capture global consistency by enforcing constraints on the overall configuration of the random variables.

3. Probabilistic Representation:

MRFs provide a probabilistic representation of the image data, enabling the estimation of the most likely segmentation or restoration.

MRF Energy Function

The central concept of MRFs is the energy function, which represents the cost associated with a particular configuration of the random variables. The energy function typically consists of two terms:

1. Data term:

This term measures the discrepancy between the image data and the current segmentation or restoration.

2. Smoothness term:

This term encourages spatial consistency by penalizing configurations with abrupt changes in pixel values.

MRF Inference Algorithms

MRF inference involves finding the configuration of random variables that minimizes the energy function. This optimization problem can be solved using various algorithms, including:

1. Simulated annealing:

This algorithm simulates a physical annealing process to gradually converge to the global minimum of the energy function.

2. Graph cuts:

This algorithm converts the MRF into a graph and finds the minimum cut of the graph, which corresponds to the optimal configuration of random variables.

3. Message passing algorithms:

These algorithms iteratively exchange messages between neighboring pixels to converge to the optimal configuration.

Applications of MRFs in Computer Vision:

MRFs have found diverse applications in various computer vision tasks:

1. Image Segmentation:

MRFs are widely used for image segmentation, effectively capturing object boundaries and handling complex image structures.

2. Image Restoration:

MRFs can be used to restore degraded images by removing noise or inpainting missing regions, guided by the underlying image structure and spatial context.

3. Image Registration:

MRFs can be used to align images with different sizes or scales by finding the transformation that minimizes the energy function, ensuring spatial consistency.

4. Texture Synthesis:

MRFs can be used to synthesize textures by learning the statistical patterns of a given texture and generating new samples with similar characteristics.

5. Stereo Vision:

MRFs can be used to compute disparity maps in stereo vision by finding the correspondence between pixels in two images that minimizes the energy function, enabling 3D reconstruction.

MRFs have emerged as a powerful and versatile tool for image analysis and computer vision. Their ability to capture spatial context, model complex relationships between pixels, and provide probabilistic representations makes them well-suited for a wide range of tasks. As computer vision algorithms continue to evolve, MRFs are likely to remain a valuable tool for image understanding and processing.

2.11 TEXTURE SEGMENTATION

Texture segmentation is a fundamental task in computer vision that aims to partition an image into distinct regions based on their texture characteristics. Texture refers to the visual pattern of an image that is characterized by the repetition of similar elements, such as grains, bumps, or ripples. The goal of texture segmentation is to identify and separate regions with different texture patterns, providing insights into the image's content and structure.

Why Texture Segmentation is Important

Texture segmentation plays a crucial role in various computer vision applications, including:

1. **Object Recognition:** Identifying and classifying objects in images by recognizing their distinct textures, such as the wood grain of a table or the fur of an animal.
2. **Medical Image Analysis:** Segmenting tissues and organs in medical images based on their texture patterns, aiding diagnosis and treatment planning.
3. **Satellite Image Analysis:** Identifying different land cover types in satellite images based on their textures, such as forests, grasslands, or water bodies.
4. **Content-Based Image Retrieval:** Retrieving images from large databases based on their texture similarity, enabling efficient search and organization of images.

5. **Image Editing and Manipulation:** Selecting and modifying specific textures in images, such as removing unwanted textures or enhancing visual appearance.

Common Texture Segmentation Techniques

Various techniques have been developed for texture segmentation, each with its strengths and limitations. Some common approaches include:

1. **Statistical Methods:** These methods analyze the statistical properties of texture, such as mean, variance, or local entropy, to identify regions with different statistical characteristics.
2. **Structural Methods:** These methods focus on the spatial arrangement of texture elements, such as orientation, periodicity, or randomness, to segment regions with distinct structural patterns.

Model-Based Methods: These methods employ mathematical models of texture, such as Gabor filters or wavelet decompositions, to capture and analyze texture features for segmentation.

Machine Learning Approaches: These approaches utilize machine learning algorithms, such as support vector machines or random forests, to learn texture patterns from labeled data and perform segmentation.

Challenges in Texture Segmentation

Texture segmentation is a challenging task due to the inherent variability and complexity of texture patterns. Some of the challenges include:

1. **Scale Invariance:** Texture appearance can vary significantly across scales, making it difficult to identify consistent texture patterns across different scales.
2. **Noise and Illumination:** Noise and varying illumination conditions can distort texture patterns and hinder accurate segmentation.
3. **Rotation and Deformation:** Textures can be rotated or deformed in images, making it difficult to recognize and segment them consistently.

4. **Subjective Perception:** Human perception of texture is subjective and context-dependent, making it challenging to define and measure texture features objectively.

5. Applications of Texture Segmentation in Computer Vision

6. Texture segmentation has found wide-ranging applications in various computer vision domains:

1. **Object Detection and Classification:** Identifying and categorizing objects in images based on their texture patterns, such as recognizing fruits, flowers, or fabric types.
2. **Medical Image Analysis:** Segmenting tissues and organs in medical images based on their texture characteristics, aiding in disease diagnosis, treatment planning, and surgical guidance.
3. **Satellite Image Analysis:** Classifying land cover types and identifying regions of interest in satellite images based on texture patterns, such as distinguishing between forests, urban areas, and water bodies.
4. **Content-Based Image Retrieval:** Enabling efficient search and organization of large image databases by retrieving images with similar texture characteristics to a given query image.
5. **Image Editing and Manipulation:** Selectively modifying textures in images, such as removing unwanted textures, enhancing visual appearance, or creating seamless texture transitions.

Texture segmentation plays a vital role in computer vision, enabling the analysis and understanding of visual patterns in images. The development of robust and efficient texture segmentation algorithms is crucial for a wide range of applications, from object recognition and medical image analysis to content-based image retrieval and image editing. As computer vision continues to evolve, texture segmentation will remain an

essential tool for extracting meaningful information from images and unlocking their potential for various applications.

UNIT III

IMAGES, HISTOGRAMS, BINARY VISION

3.1 SIMPLE PINHOLE CAMERA MODEL

The camera is one of the most essential tools in computer vision. It is the mechanism by which we can record the world around us and use its output - photographs - for various applications.

Pinhole cameras:

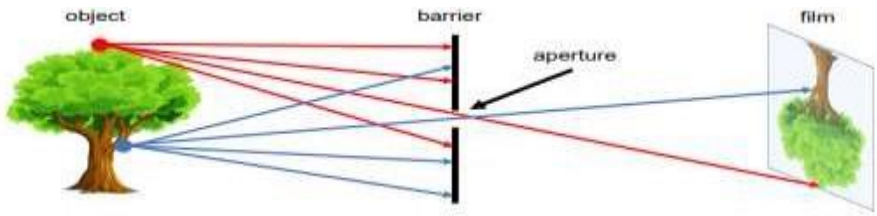


Fig 3.1 A simple working camera model: the pinhole camera model

Let's design a simple camera system – a system that can record an image of an object or scene in the 3D world. This camera system can be designed by placing a barrier with a small aperture between the 3D object and a photographic film or sensor. As Figure 1 shows, each point on the 3D object emits multiple rays of light outwards. Without a barrier in place, every point on the film will be influenced by light rays emitted from every point on the 3D object. Due to the barrier, only one (or a few) of these rays of light passes through the aperture and hits the film. Therefore, we can establish a one to-one mapping between points on the 3D object and the film. The result is that the film gets exposed by an "image" of the 3D object by means of this mapping. This simple model is known as the pinhole camera model.

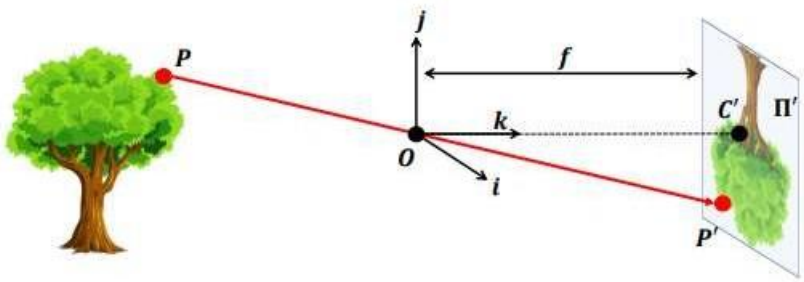
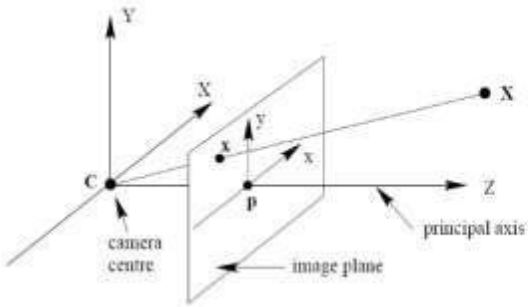


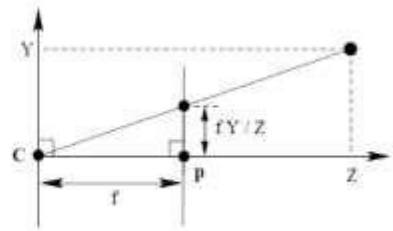
Fig 3.2 A formal construction of the pinhole camera model.

A more formal construction of the pinhole camera is shown in Figure 2. In this construction, the film is commonly called the image or retinal plane. The aperture is referred to as the pinhole O or center of the camera. The distance between the image plane and the pinhole O is the focal length f. Sometimes, the retinal plane is placed between O and the 3D object at a distance f from O. In this case, it is called the virtual image or virtual retinal plane. Note that the projection of the object in the image plane and the image of the object in the virtual image plane are identical up to a scale (similarity) transformation.

The simplest camera model is pinhole model which decribes the mathematical relationship of the projection of points in 3d-space onto a image plane. Let the centre of projection be the origin of a Euclidean coordinate system, and the plane Z=f , which is called the *image plane* or *focal plane*. Under pinhole camera model, a point in space with coordinates $(X, Y, Z)^T$ is mapped to the point on the image plane $(\frac{fX}{Z}, \frac{fY}{Z}, f)^T$ using triangles as shown in figure below. Ignoring the final image coordinate, the central projection mapping from 3d world space to 2d image coordinates is, camera geometry.

$$(X, Y, Z)^T \longrightarrow (\frac{fX}{Z}, \frac{fY}{Z})^T \quad \rightarrow \textbf{(1)}$$





The centre of projection is called the *camera centre* or the *optical centre*. The line from the camera centre perpendicular to the image plane is called the *principal axis* or *principal ray*. The point where the principal axis meets the image plane is called the *principal point*. The plane through the camera centre parallel to the image plane is called the *principal plane of the camera*. **C** is the camera centre and **p** the principal point. The camera centre is here placed at the coordinate origin [Hartley and Zisserman, 2003]. Assuming the world and image points are represented in homogeneous coordinates, then central projection can simply expressed as a linear mapping between their homogeneous coordinates in terms of matrix multiplication by,

$$\begin{bmatrix} fX \\ fY \\ Z \end{bmatrix} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} X_{cam} \\ Y_{cam} \\ Z_{cam} \\ 1 \end{bmatrix} \rightarrow \textbf{(2)}$$

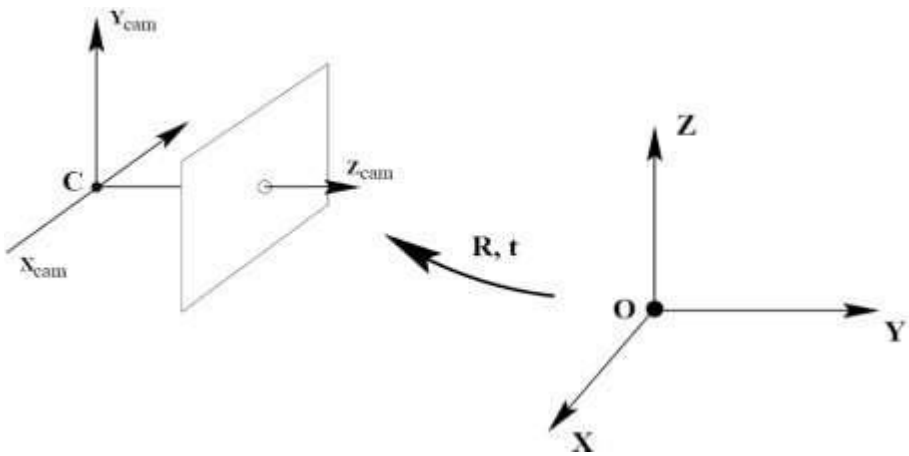
Principal point offset: In theory the origin of coordinates in the image plane assumed to be at the principal point. This may not be true in practice, hence, the Eq. (2) is express as,

$$\begin{bmatrix} fX + Zp_x \\ fY + Zp_y \\ Z \end{bmatrix} = \begin{bmatrix} f_x & 0 & p_x & 0 \\ 0 & f_y & p_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} X_{cam} \\ Y_{cam} \\ Z_{cam} \\ 1 \end{bmatrix} \rightarrow \textbf{(3)}$$

First matrix in the right side of Eq. (3) called camera *calibration matrix*, usually expressed by K. For added generality, the calibration matrix can be express as,

$$K = \begin{bmatrix} \alpha_x & s & p_x \\ 0 & \alpha_y & p_y \\ 0 & 0 & 1 \end{bmatrix} \quad \text{-→ (4)}$$

where s is referred to as the skew parameter which is zero for most of the cameras. fx and fy where alphax=fmx and αy=fmy represent the focal length of the camera in terms of pixel dimensions in the *x-axis* and the *y-axis* respectively, and (px,py) is coordinate of the principal point.



Camera rotation and translation: The Subscripts *cam* in above equations is to emphasize that the points are represented as *camera coordinate frame*. In general, points in space is determined by the *world coordinate frame*. The camera coordinate and world co

ordinate frames are related by *rotation* and *translation*. As it is shown in above figure, if X=(X,Y,Z,1) is the coordinate of the point in the world coordinates, then Xcam is transformed by,

$$\mathbf{X}_{cam} = \begin{bmatrix} R & t \end{bmatrix} \mathbf{X} \quad \text{-→ (5)}$$

where **R** is 3×3 rotation matrix and **t** is 3×1 translation matrix.

Putting eveything together the formula for general mapping of pinhole camera in world coordinate frame **x** is defined by,

$$\mathbf{x} = \mathbf{K}[\mathbf{R} \quad \mathbf{t}]\mathbf{X} \quad \text{-→ (6)}$$

So that, the general pinhole camera matrix, **P**, can be represented by,

$$\mathbf{P} = \mathbf{K}[\mathbf{R} \quad \mathbf{t}] = \begin{bmatrix} f_x & 0 & p_x \\ 0 & f_y & p_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_1 & r_2 & r_3 & t_1 \\ r_4 & r_5 & r_6 & t_2 \\ r_7 & r_8 & r_8 & t_3 \end{bmatrix} \quad \text{-→ (7)}$$

and it has 9 degrees of freedom,

1. Three for **K**, namely, f, px, and py
2. Three for rotation matrix **R**
3. Three for translation **t**

Internal camera parameters, K , show the internal orientation of the camera and it is fixed.

External parameters, R and t show camera orientation and position to a world coordinate system

3.2 SAMPLING:

Conversion of analog signal to digital signal:

The output of most of the image sensors is an analog signal, and we can not apply digital processing on it because we can not store it. We can not store it because it requires infinite memory to store a signal that can have infinite values.So we have to convert an analog signal into a digital signal.

To create an image which is digital, we need to covert continuous data into digital form. There are two steps in which it is done.

- Sampling
- Quantization

Basic idea:

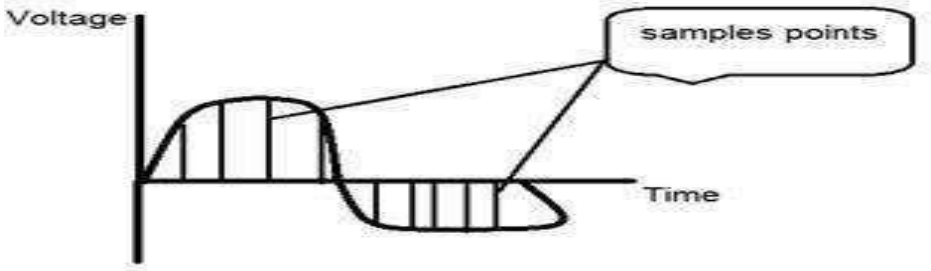
The basic idea behind converting an analog signal to its digital signal is to convert both of its axis (x,y) into a digital format.

Since an image is continuous not just in its co-ordinates (x axis), but also in its amplitude (y axis), so the part that deals with the digitizing of co-ordinates is known as sampling. And the part that deals with digitizing the amplitude is known as quantization.

Sampling:

Sampling has already been introduced in our tutorial of introduction to signals and system. But we are going to discuss here more.

- Here what we have discussed of the sampling.
- The term sampling refers to take samples
- We digitize x axis in sampling
- It is done on independent variable
- In case of equation y = sin(x), it is done on x variable
- It is further divided into two parts , up sampling and down sampling



If you will look at the above figure, you will see that there are some random variations in the signal. These variations are due to noise. In sampling we reduce this noise by taking

samples. It is obvious that more samples we take, the quality of the image would be more better, the noise would be more removed and same happens vice versa.

However, if you take sampling on the x axis, the signal is not converted to digital format, unless you take sampling of the y-axis too which is known as quantization. The more samples eventually means you are collecting more data, and in case of image, it means more pixels.

Relation ship with pixels

Since a pixel is a smallest element in an image. The total number of pixels in an image can be calculated as

Pixels = total no of rows * total no of columns.

Lets say we have total of 25 pixels, that means we have a square image of 5 X 5. Then as we have dicussed above in sampling, that more samples eventually result in more pixels. So it means that of our continuous signal, we have taken 25 samples on x axis. That refers to 25 pixels of this image.

This leads to another conclusion that since pixel is also the smallest division of a CCD array. So it means it has a relationship with CCD array too, which can be explained as this.

Relationship with CCD array

The number of sensors on a CCD array is directly equal to the number of pixels. And since we have concluded that the number of pixels is directly equal to the number of samples, that means that number sample is directly equal to the number of sensors on CCD array.

Oversampling:

In the beginning we have define that sampling is further categorize into two types. Which is up sampling and down sampling. Up sampling is also called as over sampling.

The oversampling has a very deep application in image processing which is known as Zooming.

Zooming:

We will formally introduce zooming in the upcoming tutorial, but for now on, we will just briefly explain zooming.

Zooming refers to increase the quantity of pixels, so that when you zoom an image, you will see more detail.

The increase in the quantity of pixels is done through oversampling. The one way to zoom is, or to increase samples, is to zoom optically, through the motor movement of the lens and then capture the image. But we have to do it, once the image has been captured.

There is a difference between zooming and sampling

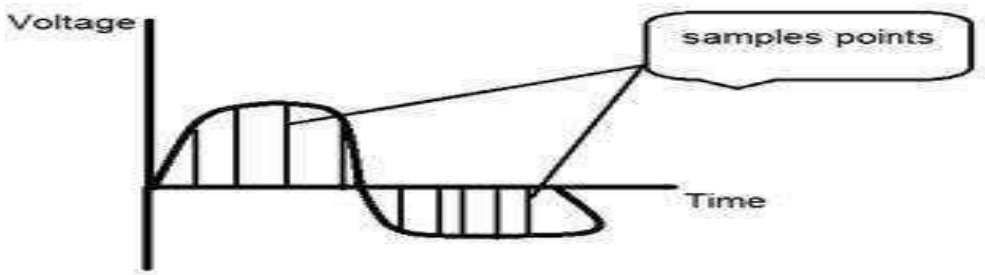
The concept is same, which is, to increase samples. But the key difference is that while sampling is done on the signals, zooming is done on the digital image.

3.3 QUANTIZATION

Digitizing a signal

As we have seen in the previous tutorials, that digitizing an analog signal into a digital, requires two basic steps. Sampling and quantization. Sampling is done on x axis. It is the conversion of x axis (infinite values) to digital values.

The below figure shows sampling of a signal.



Sampling with relation to digital images

The concept of sampling is directly related to zooming. The more samples you take, the more pixels, you get. Oversampling can also be called as zooming. This has been discussed under sampling and zooming tutorial.

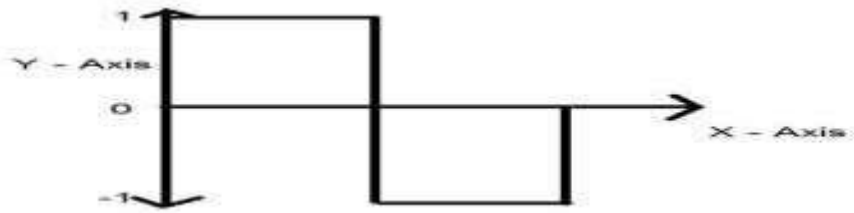
But the story of digitizing a signal does not end at sampling too, there is another step involved which is known as Quantization.

What is quantization

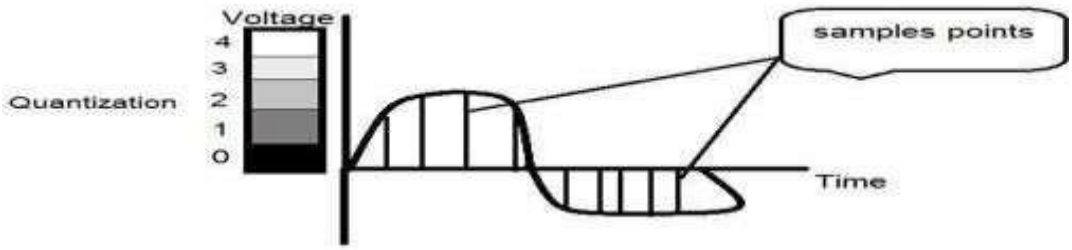
Quantization is opposite to sampling. It is done on y axis. When you are quantizing an image, you are actually dividing a signal into quanta(partitions).

On the x axis of the signal, are the co-ordinate values, and on the y axis, we have amplitudes. So digitizing the amplitudes is known as Quantization.

Here how it is done



You can see in this image, that the signal has been quantified into three different levels. That means that when we sample an image, we actually gather a lot of values, and in quantization, we set levels to these values. This can be more clear in the image below.

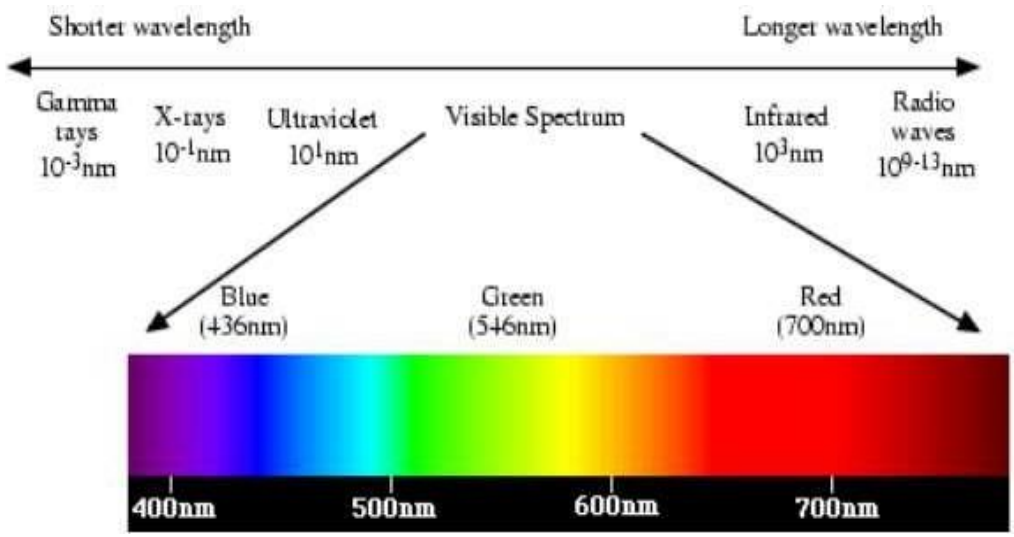


In the figure shown in sampling, although the samples has been taken, but they were still spanning vertically to a continuous range of gray level values. In the figure shown above, these vertically ranging values have been quantized into 5 different levels or partitions. Ranging from 0 black to 4 white. This level could vary according to the type of image you want.

The relation of quantization with gray levels has been further discussed below.

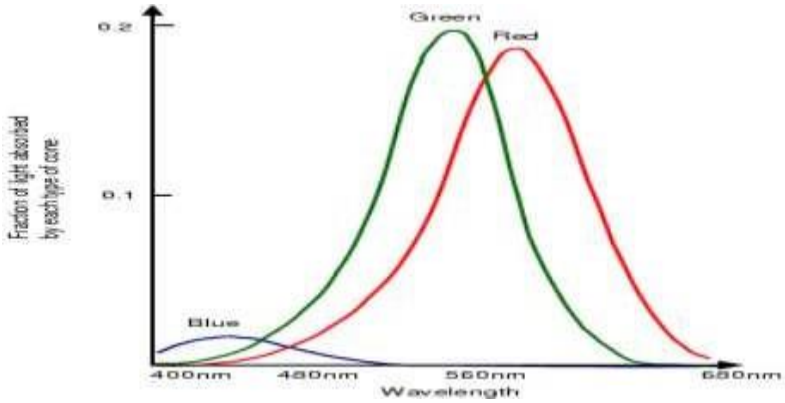
The human visual system can distinguish hundreds of thousands of different colour shades and intensities, but only around 100 shades of grey. Therefore, in an image, a great deal of extra information may be contained in the colour, and this extra information can then be used to simplify image analysis, e.g. object identification and extraction based on colour.

Three independent quantities are used to describe any particular colour. The *hue* is determined by the dominant wavelength. Visible colours occur between about 400nm (violet) and 700nm (red) on the electromagnetic spectrum, as shown in figure 1.3



3.4.1 Tristimulus theory of colour perception

As discussed in lecture 1, the human retina has 3 kinds of cones. The response of each type of cone as a function of the wavelength of the incident light is shown in figure 2. The peaks for each curve are at 440nm (blue), 545nm (green) and 580nm (red). Note that the last two actually peak in the yellow part of the spectrum[2].



3.4 NOISE

One challenge when using digital cameras is something called noise. That's when individual pixels in the image appear brighter or darker than they should be, due to interference in the electronic circuits inside the camera. It's more of a problem when light levels are dark, and the camera tries to boost the exposure of the image so that you can see more. You can see this if you take a digital photo in low light, and the camera uses a high ASA/ISO setting to capture as much light as possible. Because the sensor has been made very sensitive to light, it is also more sensitive to random interference, and gives photos a "grainy" effect.

Noise mainly appears as random changes to pixels. For example, the following image has "salt and pepper" noise.

Having noise in an image can make it harder to recognise what's in the image, so an important step in computer vision is reducing the effect of noise on an image. There are well-understood techniques for this, but they have to be careful that they don't discard useful information in the process. In each case, the technique has to make an educated

guess about the image to predict which of the pixels that it sees are supposed to be there, and which aren't.

This helps to reduce the level of noise in the image. Can you tell why, and about how much less noise there might be? (As an experiment, you could take a photo in low light – can you see small patches on it caused by noise? Now use photo editing software to change it to black and white – does that reduce the effect of the noise?)

Rather than just considering the red, green and blue values of each pixel individually, most noise-reduction techniques look at other pixels in a region, to predict what the value in the middle of that neighbourhood ought to be.

However, given the fact that noise, edges and textures are all high-frequency elements, simply applying filters to eliminate these frequencies removes noise as well as desirable aspects that need to be preserved. Some major challenges include:

- Ensuring that flat areas remain smooth
- Ensuring that edges are protected without blurring
- Ensuring that textures are preserved
- Ensuring that new artifacts are not generated

Classic Denoising

Spatial domain methods aim to remove noise by calculating the grey value of each pixel based on the correlation between pixels and image patches in the original image. The grey value of a pixel is defined as its brightness, with the minimum value being 0 and the maximum value being dependent on the digitization depth of the image. For an 8-bit depth, this maximum value is 255.

Transform techniques

In contrast with spatial methods, where denoising is performed on the actual image, transform techniques, as the name suggests, first 'translate' the given image into another domain by performing a set of mathematical operations known as transformations.

In extremely simple terms, what this means is that the algorithms are attempting to learn the relationship between the two sets of images. Once learned, they would be able to automatically apply it to similarly noisy images or videos to produce the denoised versions.

3.5 SMOOTHING

Smoothing in image processing is a technique used to reduce noise and fine details in an image by applying a low-pass filter. This filter works by replacing each pixel value with an average value of its neighboring pixels. Smoothing can help to improve the visual quality of an image and make it easier to analyze by reducing the impact of small variations in pixel values. However, too much smoothing can result in the loss ofimportant information, so it's important to choose an appropriate level of smoothing basedon the specific requirements of the application.

Smoothing in image processing refers to the process of reducing noise or other unwanted artifacts in an image while preserving important features and structures. The goal of smoothing is to create a visually appealing image that is easy to interpret and analyze. Smoothing techniques use various algorithms, such as filters or convolutions, to remove noise or other distortions in the image. Effective smoothing requires striking a balance between removing unwanted artifacts and preserving important image details, and is an essential step in many image processing applications, including **image segmentation, object recognition, and computer vision.**

3.6.1 Types of Smoothing Filters

Smoothing filters, also known as **blurring filters**, are a type of image filter that are commonly used in image processing to reduce noise and remove small details from an image. There are several types of smoothing filters, including mean filter, median filter, Gaussian filter, and bilateral filter.

Wavelet-based techniques are often used for image compression and denoising, as they can preserve important features while reducing the size of the image or removing unwanted noise.

Applications of Smoothing

Smoothing, also known as image filtering or image smoothing, is a technique used to remove noise and other unwanted artifacts from images. This technique can be applied to various types of noise, including salt-and-pepper noise and Gaussian noise. Smoothing is also used for edge preservation, pre-processing, and aesthetic purposes.

Noise Reduction (e.g., Removing Salt-And-Pepper Noise, Gaussian Noise)

Smoothing is commonly used to reduce noise in images. Noise can be caused by various factors, including poor lighting, poor camera settings, and other environmental factors. Salt-and-pepper noise is a type of noise that appears as isolated pixels in an image, either completely white or completely black. **Gaussian noise**, on the other hand, is a type of noise that appears as a random pattern of light and dark pixels. To remove these types of noise, a smoothing filter can be applied to the image. This filter averages the pixel values in the image, reducing the impact of isolated pixels and random patterns. There are various types of smoothing filters, including mean filter, median filter, and Gaussian filter, each with its own strengths and weaknesses.

Edge Preservation (e.g., Smoothing an Image While Preserving Edges)

When smoothing an image, it's important to preserve the edges in the image. Edges are the boundaries between different regions in the image, such as the boundary between the sky and the mountains in a landscape photograph. If edges are lost during the smoothing process, the image can appear blurry or unclear. To preserve edges during the smoothing process, a filter can be applied that selectively smooths areas of the image that are away from edges, while leaving the edges untouched. Examples of edge-preserving filters include **bilateral filter, guided filter, and anisotropic diffusion filter**.

Pre-Processing (e.g., Preparing an Image for Further Processing)

Smoothing can also be used as a pre-processing step before applying other image-processing techniques, such as segmentation or feature extraction. For example, if an image is very noisy or has a lot of texture, it can be difficult to accurately identify objects in the image. By smoothing the image first, the **noise and texture** can be reduced, making it easier to identify objects in the image.

Aesthetic Purposes (e.g., Creating a Blurred Background Effect)

Smoothing can be used for aesthetic purposes, such as creating a **blurred background** effect in a portrait photograph. By selectively smoothing the background of the image, the subject of the photograph can be made to stand out more. Similarly, smoothing can be used to create a painterly effect in an image, giving it a more artistic look and feel.

3.6.2 Algorithms for Smoothing

Standard Algorithms (e.g., Box Filter, Median Filter)

Standard algorithms for smoothing are a set of techniques that can be used to reduce noise in signals or images. These algorithms are often simple and efficient, and they can be implemented quickly on a computer. Two common types of standard algorithms for smoothing are the box filter and the median filter.

Box filter

- The box filter, also known as the average filter, is a technique used to smooth signals or images by averaging the values in a neighborhood around each pixel. The neighborhood is usually defined by a rectangular or square window, and the size of the window determines the degree of smoothing. The box filter is a linear filter, meaning that the output is a weighted sum of the input values.

Median filter

- The median filter is a non-linear filter used to remove noise from signals or images. Instead of averaging the values in a neighborhood around each pixel, the

median filter replaces each pixel with the median value in its neighborhood. The median filter is often more effective than the box filter at removing noise while preserving edges and other features in the image.

Machine Learning-Based Algorithms (e.g., Deep Neural Networks)

Machine learning-based algorithms for smoothing are a set of techniques that use **artificial neural networks** to learn how to remove noise from signals or images. These algorithms are often more complex than standard algorithms, but they can be more effective at removing noise and preserving features in the image.

3.6 HISTOGRAMS

The histogram is used for graphical representation of a digital image. A graph is a plot by the number of pixels for each tonal value. Nowadays, image histogram is present in digital cameras. Photographers use them to see the distribution of tones captured.

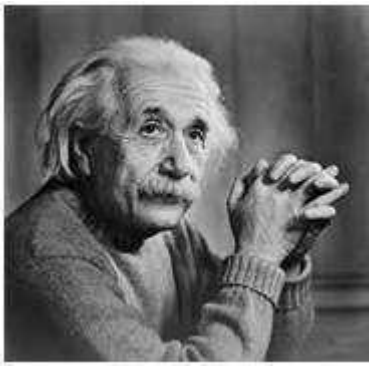
In a graph, the horizontal axis of the graph is used to represent tonal variations whereas the vertical axis is used to represent the number of pixels in that particular pixel. Black and dark areas are represented in the left side of the horizontal axis, medium grey color is represented in the middle, and the vertical axis represents the size of the area.

Histogram of the above scenery

3.7 HISTOGRAM EQUALIZATION

Histogram equalization is used to enhance contrast. It is not necessary that contrast will always be increase in this. There may be some cases were histogram equalization can be worse. In that cases the contrast is decreased.

Lets start histogram equalization by taking this image below as a simple image.



CDF

Our next step involves calculation of CDF (cumulative distributive function). Again if you do not know how to calculate CDF, please visit our tutorial of CDF calculation.

Calculate CDF according to gray levels

Lets for instance consider this, that the CDF calculated in the second step looks like this.

Global histogram equalization (GHE)

GHE is very simple and fast, but its contrast enhancement power is low. Here the histogram of the whole input image is used to compute the histogram transformation

function. As a result, the dynamic range of the image histogram is flattened and stretched. The overall contrast is improved.

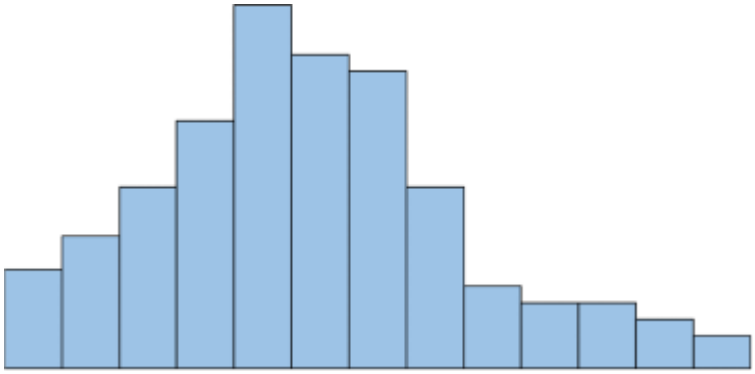
Local histogram equalization (LHE)

LHE can enhance the overall contrast more effectively.

One of the drawbacks of histogram equalization is that it can change the mean brightness of an image significantly as a consequence of histogram flattening and sometimes this is not a desirable property when preserving the original mean brightness of a given image is necessary. Bi-Histogram Equalization was proposed to overcome this problem.

3.8 HISTOGRAM COMPARISON

A **histogram** is a type of chart that allows us to visualize the distribution of values in a dataset.



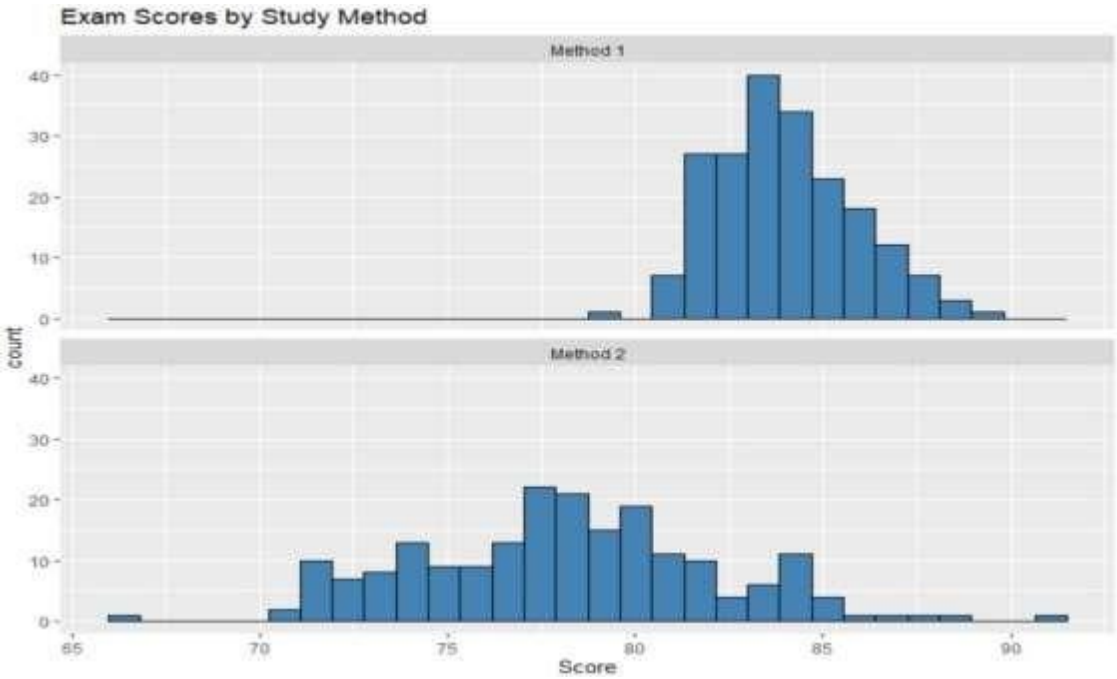
The x-axis displays the values in the dataset and the y-axis shows the frequency of each value.

Histograms are useful because they allow us to gain a quick understanding of the distribution of values in a dataset. They're also useful for comparing two different datasets.

When comparing two or more histograms, we can answer three different questions:

Suppose 200 students use one study method to prepare for an exam and another 200 students use a different study method to prepare for the same exam.

Suppose we create the following histograms to compare the exam scores for each group of students:



3.9 HISTOGRAM BACKPROJECTION

Histogram back projection is a technique used in computer vision for image segmentation and object localization.

Here's a basic overview of the histogram back projection process:

1. **Create a Model Histogram:**
    - Select a region in an image that represents the object of interest.
    - Create a histogram of the pixel values (colors) in that region.
2. **Back projection:**
    - Given a model histogram, calculate the probability of each pixel in the input image belonging to the object.
    - This is done by comparing the pixel values in the image to the model histogram.

3. **Thresholding:**
    - Apply a threshold to the back projection result to identify regions where the probability is high.
    - The thresholder image highlights regions that are likely to belong to the object.

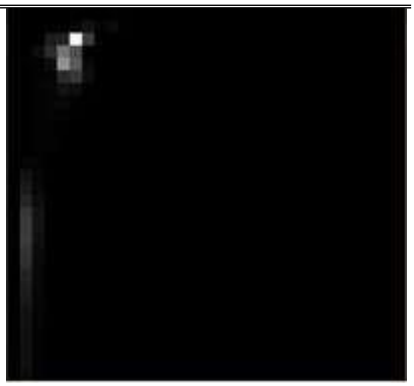
How does it work?

- We explain this by using the skin example:
- Let's say you have gotten a skin histogram (Hue-Saturation) based on the image below. The histogram besides is going to be our *model histogram* (which we know represents a sample of skin tonality). You applied some mask to capture only the histogram of the skin area:

T0 T1

- Now, let's imagine that you get another hand image (Test Image) like the one below: (with its respective histogram):

T2



T3

- What we want to do is to use our *model histogram* (that we know represents a skin tonality) to detect skin areas in our Test Image. Here are the steps

    1. In each pixel of our Test Image (i.e. p(i,j)), collect the data and find the correspondent bin location for that pixel (i.e. (hi,j,si,j)).
    2. Lookup the *model histogram* in the correspondent bin - (hi,j,si,j) - and read the bin value.
    3. Store this bin value in a new image (*Back Projection*). Also, you may consider to normalize the *model histogram* first, so the output for the Test Image can be visible for you.
    4. Applying the steps above, we get the following Back Projection image for our Test Image:

    5. In terms of statistics, the values stored in *Back Projection* represent the *probability* that a pixel in *Test Image* belongs to a skin area, based on the *model histogram* that we use. For instance in our Test image, the

brighter areas are more probable to be skin area (as they actually are), whereas the darker areas have less probability (notice that these "dark" areas belong to surfaces that have some shadow on it, which in turns affects the detection).

- Histogram Back projection, a technique that is used for image segmentation or finding objects of interest in an image. It was proposed by **Michael J. Swain, Dana H. Ballard** in their paper **Indexing via color histograms,** Third international conference on computer vision,1990.
- This was one of the first works that use color to address the classic problem of Classification and Localisation in computer vision.

3.10 K- MEANS CLUSTERING

In computer vision, image segmentation is the process of partitioning an image into multiple segments. The goal of segmenting an image is to change the representation of an image into something that is more meaningful and easier to analyze. It is usually used for locating objects and creating boundaries.

It is not a great idea to process an entire image because many parts in an image may not contain any useful information. Used in self-driving cars. Autonomous driving is not possible without object detection which involves segmentation.

K Means Clustering Algorithm:

K Means is a clustering algorithm. Clustering algorithms are unsupervised algorithms which means that there is no labelled data available. It is used to identify different classes or clusters in the given data based on how similar the data is. Data points in the same group are more similar to other data points in that same group than those in other groups.

K-means clustering is one of the most commonly used clustering algorithms. Here, k represents the number of clusters.

Let’s see how does K-means clustering work –

1. Choose the number of clusters you want to find which is k.
2. Randomly assign the data points to any of the k clusters.

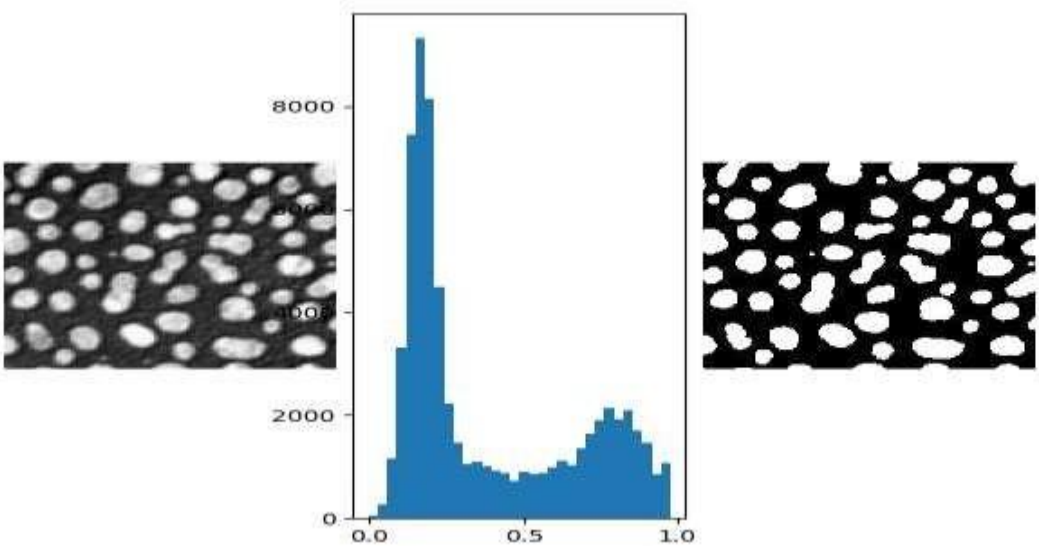
3. Then calculate the center of the clusters.
4. Calculate the distance of the data points from the centers of each of the clusters.
5. Depending on the distance of each data point from the cluster, reassign the data points to the nearest clusters.
6. Again calculate the new cluster center.
7. Repeat steps 4,5 and 6 till data points don't change the clusters, or till we reach the assigned number of iterations.

Requirements:

Make sure you have Python, Numpy, Matplotlib and OpenCV installed.

3.11 THRESHOLDING

Thresholding is the simplest method of segmenting images. It plays a crucial role in image processing as it allows for the segmentation and extraction of important information from an image. By dividing an image into distinct regions based on pixel intensity or pixel value, thresholding helps distinguish objects or features of interest from the background. This technique is widely used in various applications such as object detection, image segmentation, and character recognition, enabling efficient analysis and interpretation of digital images. Additionally, image thresholding can enhance image quality by reducing noise and improving overall visual clarity.



What is Image Thresholding?

Image thresholding involves dividing an image into two or more regions based on intensity levels, allowing for easy analysis and extraction of desired features. By setting a threshold value, pixels with intensities above or below the threshold can be classified accordingly This technique aids in tasks such as object detection, segmentation, and image enhancement.

Image Thresholding Techniques

3.12 MATHEMATICAL MORPHOLOGY IN COMPUTER VISION

Mathematical morphology is a theory and technique for the analysis and processing of geometrical structures, especially shapes and patterns in images. It's widely used in computer vision for tasks like image preprocessing, object extraction, and shape analysis. Mathematical morphology operates on binary and grayscale images using set-theoretic operations.

Key concepts in mathematical morphology include structuring elements (also known as kernels or masks) and operations like dilation, erosion, opening, and closing. Here's an overview of these concepts:

1. **Structuring Element:**
    - A structuring element is a small, typically binary image used as a reference for the morphological operations.
    - It defines the neighborhood around each pixel and influences the operation's behavior.
    - Common shapes for structuring elements include squares, rectangles, circles, and custom shapes.
2. **Dilation:**
    - Dilation is an operation that expands the boundaries of objects in a binary image.
    - For each pixel in the input image, the corresponding pixel in the output image is set to 1 if at least one pixel in the structuring element is 1 in the

neighborhood.

3. **Erosion:**
    - Erosion is an operation that shrinks the boundaries of objects in a binary image.
    - For each pixel in the input image, the corresponding pixel in the output image is set to 1 only if all pixels in the structuring element are 1 in the neighborhood.
4. **Opening:**
    - Opening is a sequence of erosion followed by dilation.
    - It is useful for removing small objects and smoothing the contours of larger objects.
5. **Closing:**
    - Closing is a sequence of dilation followed by erosion.
    - It is effective in closing small holes and gaps in objects.

Mathematical morphology is not limited to binary images; it can also be applied to grayscale images by defining structuring elements and operations based on intensity values.

Generally speaking most morphological operations are based on simple expanding and shrinking operations. The primary application of morphology occurs in binary images, though it is also used on grey level images. It can also be useful on range images. (A range image is one where grey levels represent the distance from the sensor to the objects in the scene rather than the intensity of light reflected from them).

Set operations

The two basic morphological set transformations are *erosion* and *dilation*

These transformations involve the interaction between an image *A* (the object of interest) and a structuring set *B*, called the *structuring element*.

Typically the structuring element *B* is a circular disc in the plane, but it can be any shape. The image and structuring element sets need not be restricted to sets in the 2D plane, but could be defined in 1, 2, 3 (or higher) dimensions.

$$A_x = \{c : c = a + x, \text{for } a \in A\}.$$

The *reflection* of *B*, denoted $\hat{B}$, is defined as

$$\hat{B} = \{x : x = -b, \text{for } b \in B\}.$$

The complement of *A* is denoted A^c, and the difference of two sets *A* and *B* is denoted *A* - *B*.

Dilation

Dilation of the object *A* by the structuring element *B* is given by

$$A \oplus B = \{x : \hat{B}_x \cap A \neq \emptyset\}.$$

The result is a new set made up of all points generated by obtaining the reflection of *B* about its origin and then shifting this relection by *x*.

Consider the example where *A* is a rectangle and *B* is a disc centred on the origin. (Note that if *B* is not centred on the origin we will get a translation of the object as well.) Since *B* is symmetric, $\hat{B} = B$.

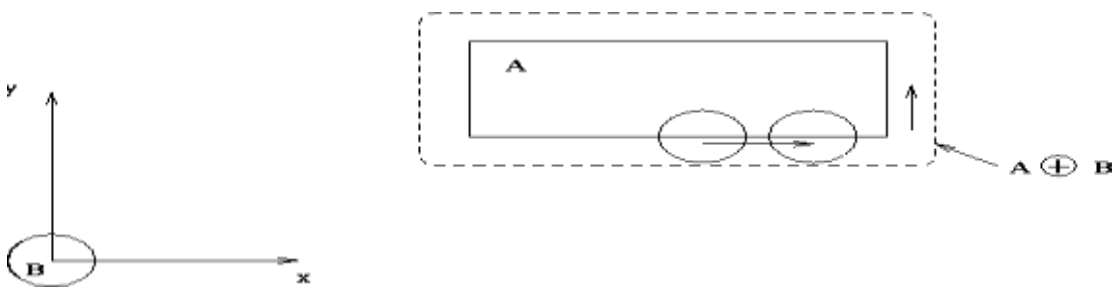


Figure : *A* is dilated by the structuring element *B*.

This definition becomes very intuitive when the structuring element *B* is viewed as a convolution mask.

where $X_0 = p$, and B is the `cross' structuring element shown in figure 8. The algorithm terminates when $X_k = X_{k-1}$. The set union of X_k and A contains the filled set and its boundary.

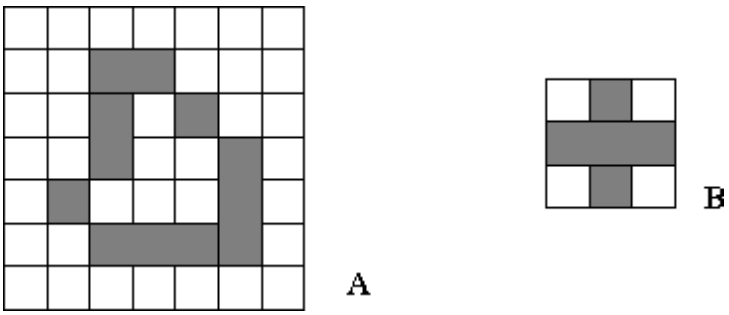


Figure : The region in A is filled using the structuring element B.

Likewise, *connected components* can also be extracted using morphological operations. If Y represents a connected component in an image A and a point p in Y is known, then the following iterative expression yields all the elements of Y:

$$X_k = (X_{k-1} \oplus B) \cap A, \text{for } k = 1, 2, \ldots$$

where $X_0 = p$ and B is a 3×3 matrix of 1s. If $X_k = X_{k-1}$ the algorithm has converged and we let $Y = X_k$.

An important step in representing the structural shape of a planar region is to reduce it to a graph. This is very commonly used in robot path planning. This reduction is most commonly achieved by reducing the region to its *skeleton*.

The skeleton of a region is defined by the medial axis transformation (MAT). The MAT of a region R with border B is defined as follows: for each point p in R, we find its closest neighbour in B. If p has more than one such closest neighbour, then p belongs to the medial axis (or skeleton) of R. Of course, closest depends on the metric used. Figure 9 shows some examples with the usual Euclidean metric.

$$S_k(A) = \bigcup_{k=0}^{K} \{(A \ominus kB) - [(A \ominus kB) \circ B],$$

3.13 CONNECTIVITY

In computer vision, connectivity refers to the way pixels or image components are connected to form meaningful structures. It is a fundamental concept in image analysis and plays a crucial role in various tasks such as object recognition, segmentation, and feature extraction. Connectivity can be defined in different ways, and it depends on the context of the problem being addressed. Here are some key aspects of connectivity in computer vision:

1. **Pixel Connectivity:**
    - **4-Connected:** In a 4-connected image, pixels are considered connected if they share a common edge. Two pixels are 4-connected if they are neighbors to the north, south, east, or west of each other.
    - **8-Connected:** In an 8-connected image, pixels are considered connected if they share a common edge or corner. Two pixels are 8-connected if they are neighbors in any direction.
2. **Component Connectivity:**
    - In the context of connected components, connectivity refers to the way pixels form meaningful groups or objects.
    - **Connected Components:** A connected component is a set of connected pixels with the same label or intensity. Connectivity can influence how components are defined and segmented in an image.
3. **Region Connectivity:**
    - In image segmentation, connectivity is often used to define regions based on pixel relationships.
    - **Region Growing:** A region-growing algorithm starts with a seed pixel and adds neighboring pixels that satisfy a certain criterion (e.g., similar

intensity or color). The connectivity criteria influence how the region grows.

4. **Graph Connectivity:**
   - In graph-based representations of images, connectivity defines the relationships between pixels or image regions.
   - **Graph Cut:** Graph cut algorithms use connectivity information to segment an image into regions. Edges in the graph represent the strength of connectivity between pixels or regions.

5. Skeletonization and Thinning:

- Connectivity is crucial in skeletonization and thinning processes, where the goal is to reduce the binary object to its essential connectivity structure while preserving its shape.

Understanding and leveraging connectivity in computer vision algorithms can help improve the accuracy and robustness of image processing tasks. The choice of connectivity criteria depends on the specific requirements of the application and the characteristics of the images being analyzed.

Pixel Connectivity

The notation of pixel connectivity describes a relation between two or more pixels. For two pixels to be connected they have to fulfill certain conditions on the pixel brightness and spatial adjacency.

First, in order for two pixels to be considered connected, their pixel values must both be from the same set of values *V*. For a grayscale image, *V* might be any range of gray levels, *e.g. V={22,23,...40}*, for a binary image we simple have *V={1}*.

To formulate the adjacency criterion for connectivity, we first introduce the notation of *neighborhood*. For a pixel *p* with the coordinates *(x,y)* the set of pixels given by:

$$N_4(p) = \{(x+1, y), (x-1, y), (x, y+1), (x, y-1)\}$$

is called its *4-neighbors*. Its *8-neighbors* are defined as

UNIT IV

3D VISION AND MOTION

Methods for 3D vision – projection schemes – shape from shading – photometric stereo – shape from texture – shape from focus – active range finding – surface representations – point-based representation – volumetric representation – 3D object recognition – 3D reconstruction – introduction to motion – triangulation – bundle adjustment – translational alignment – parametric motion – spline-based motion – optional flow – layered motion.

4.1 Methods for 3D vision

This chapter provides a reminder of main 3D vision methods of interest for macro and micro range imaging.

Depth from focus – Multi focus 3D microscopy

- **Stereo vision**
- **Structured light**
- **Chromatic aberration**
- **White light interferometry**
- **Confocal microscopy**

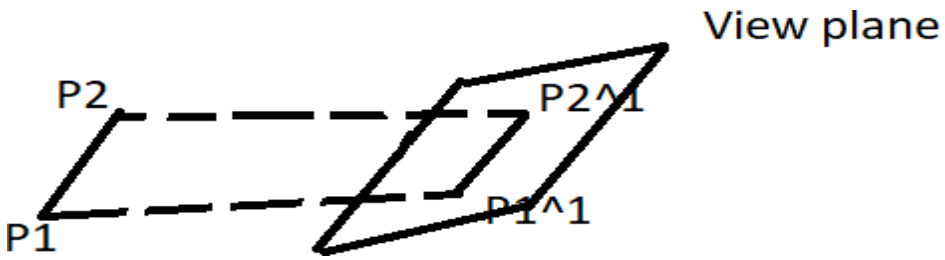


Figure 3.2 Parallel Projection

Parallel projection is divided into two parts and these two-parts sub divided into many.

Orthographic Projections:

In orthographic projection the direction of projection is normal to the projection of the plane. In orthographic lines are parallel to each other making an angle 90 with view plane. Orthographic parallel projections are done by projecting points along parallel lines that are perpendicular to the projection line. Orthographic projections are most often used to procedure the front, side, and top views of an object are called evaluations.

- **Isometric Projections:** Orthographic projections that show more than one side of an object are called axonometric orthographic projections. The most common axonometric projection is an isometric projection. In this projection parallelism of lines are preserved but angles are not preserved.
- **Dimetric projections:** In these two projectors have equal angles with respect to two principal axis.
- **Trimetric projections:** The direction of projection makes unequal angle with their principal axis.

- Integrated, GenICam compatible sequencer for controlling illumination and triggering image acquisition directly from the camera
- No external illumination controller requirement saves cost and time by reducing cabling and programming effort
- Reliable detection of even the slightest surface irregularities with up to 65 megapixel resolution

4.1.2 Photometric stereo

Photometric stereo is a technique in computer vision for estimating the surface normals of objects by observing that object under different lighting conditions (photometry). It is based on the fact that the amount of light reflected by a surface is dependent on the orientation of the surface in relation to the light source and the observer. By measuring the amount of light reflected into a camera, the space of possible surface orientations is limited.

To produce a unified model, the paper takes the largest partial model, and incrementally aligns all smaller partial models to it. Global minimization is used to reduce the error, then a Euclidean upgrade is used to change the model's feature positions from 3D coordinates unique up to affine transformation to 3D coordinates that are unique up to Euclidean motion. At the end of this step, one has a model of the target object, consisting of features projected into a common 3D space.

1. **Parametric 3D Models:**

   In applications involving 3D computer vision, parametric models may represent the motion and shape of objects in three-dimensional space. Parameters could include translations, rotations, and other transformations that describe the object's motion.

2. **Parametric Representations in Neural Networks:**

   With the rise of deep learning, neural networks can be trained to learn parametric representations of motion. Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks are examples of architectures that can capture temporal dependencies and model motion over sequential data.

Parametric motion models are essential for tasks such as video analysis, object tracking, and action recognition in computer vision. The choice of the model depends on the specific characteristics of the motion in the given application.

4.3.1 Spline-Based Motion:

In the context of computer vision, splines can be employed to model the trajectory or path of objects in motion. The motion of objects in a video sequence can be complex and non-linear, and splines provide a flexible way to represent such motion. Different types of splines, such as B-splines or cubic splines, can be used depending on the specific requirements of the application.

Here are some ways spline-based motion is applied in computer vision:

1. **Object Tracking:** Splines can be used to model the trajectory of an object over time. By fitting a spline curve to the observed positions of an object in consecutive frames, you can obtain a smooth representation of its motion. This can be useful for tracking

objects in videos.

2. Interpolation and Prediction: Splines can be used to interpolate between known points in a sequence, providing a smooth estimate of the object's position at any given time. This is particularly useful for predicting the future positions of moving objects.

3. Image Registration: In medical imaging or remote sensing, where images from different modalities or times need to be aligned, splines can be used for image registration. This involves finding a transformation that aligns images, and splines can represent the spatial deformation fields required for registration.

4. Motion Analysis: Splines can be applied to analyze and classify different types of motion. For example, they can be used to characterize the gait of a person or the movement pattern of a vehicle.

UNIT V

APPLICATIONS

Overview of Diverse Computer Vision Applications: Document Image Analysis, Biometrics, Object Recognition, Tracking, Medical Image Analysis, Content-Based Image Retrieval, Video Data Processing, Virtual Reality and Augmented Reality.

5.1 Overview of Diverse Computer Vision Applications:

Computer vision is a field of artificial intelligence that enables machines to interpret and make decisions based on visual data. It involves the development of algorithms and models that allow computers to understand and analyze images and videos. The applications of computer vision are diverse and span various industries. Here's an overview of some prominent computer vision applications:

Image Classification:

Description: Assigning predefined labels or categories to images based on their content.

Example: Identifying objects in photographs, such as distinguishing between cats and

dogs.

Object Detection:

Description: Locating and classifying objects within images or video frames.

Example: Autonomous vehicles identifying pedestrians, traffic signs, and other vehicles.

Facial Recognition:

Description: Identifying and verifying individuals based on facial features.

Example: Unlocking smartphones, airport security, and law enforcement applications.

Gesture Recognition:

Description: Understanding and interpreting human gestures from image or video data.

Example: Virtual reality interactions, sign language recognition.

OCR (Optical Character Recognition):

Description: Extracting text information from images or documents.

Example: Converting scanned documents into editable text, license plate recognition.

Medical Image Analysis:

Description: Analyzing medical images, such as X-rays, MRIs, and CT scans, for diagnosis and treatment planning.

Example: Tumor detection, organ segmentation, pathology analysis.

Video Analysis:

Description: Extracting information from video streams.

Example: Surveillance systems, action recognition, anomaly detection.

Augmented Reality (AR):

Description: Overlapping virtual information on the real-world view.

Example: AR gaming, navigation assistance, virtual try-on for online shopping.

Robotics:

Description: Enabling robots to perceive and interact with their environment.

Example: Robotic navigation, object manipulation, assembly line automation.

Autonomous Vehicles:

Description: Providing visual perception for self-driving cars and drones.

Example: Lane detection, obstacle avoidance, traffic sign recognition.

Industrial Inspection:

Description: Quality control and defect detection in manufacturing processes.

Example: Identifying defects on production lines, ensuring product quality.

Agricultural Monitoring:

Description: Analyzing crop health and monitoring agricultural fields.

Example: Identifying crop diseases, assessing plant growth.

Retail Analytics:

Description: Analyzing customer behavior and optimizing store layouts.

Example: People counting, shelf inventory monitoring.

Environmental Monitoring:

Description: Analyzing satellite or drone images for environmental purposes.

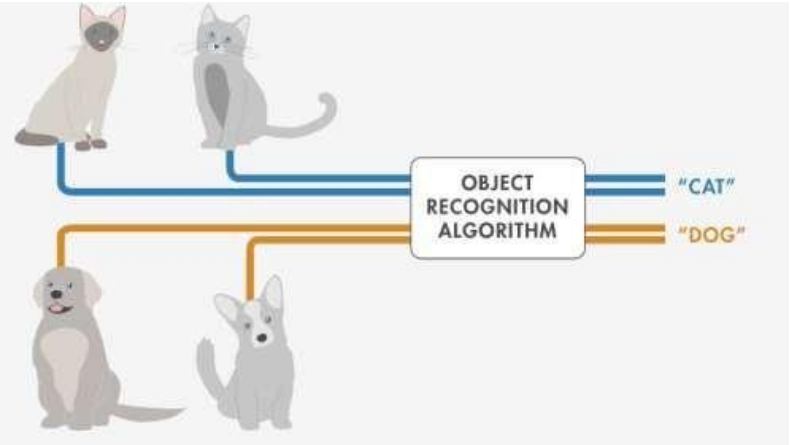
Example: Deforestation detection, pollution monitoring.

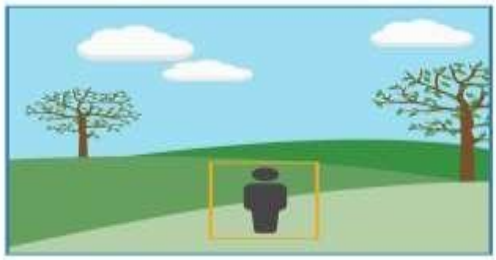
Emotion Recognition:

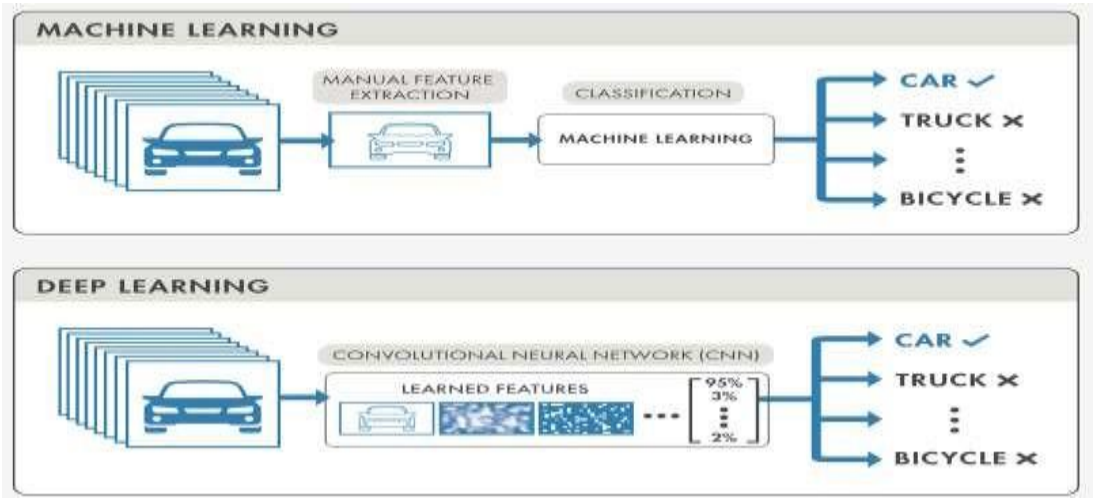
Description: Analyzing facial expressions to determine emotional states.

5.2 Object Recognition

Object recognition is a computer vision technique for identifying objects in images or videos. Object recognition is a key output of deep learning and machine learning algorithms. When humans look at a photograph or watch a video, we can readily spot people, objects, scenes, and visual details. The goal is to teach a computer to do what comes naturally to humans: to gain a level of understanding of what an image contains.







Object Recognition Techniques

Object Recognition Using Deep Learning

Deep learning techniques have become a popular method for doing object recognition. Deep learning models such as convolutional neural networks, or CNNs, are used to automatically learn an object's inherent features in order to identify that object.

- **Using a pretrained deep learning model**: Most deep learning applications use the transfer learning approach, a process that involves fine-tuning a pretrained model. You start with an existing network, such as Alex Net or GoogLe Net, and feed in new data containing previously unknown classes. This method is less time-consuming and can provide a faster outcome because the model has already been trained on thousands or millions of images.

Machine Learning Workflow

To perform object recognition using a standard machine learning approach, you start with a collection of images (or video) and select the relevant features in each image. For example, a feature extraction algorithm might extract edge or corner features that can be used to differentiate between classes in your data.

These features are added to a machine learning model, which will separate these features into their distinct categories, and then use this information when analyzing and classifying new objects.

Object tracking use cases in computer vision

There are many applications of object tracking in the real world. Here are a few notable examples of how object tracking is used:

1. **Autonomous Driving: Obstacle detection in self-driving cars**, pedestrians detection on roads and sidewalks, collision avoidance and vehicle speed detection, traffic monitoring and route estimation, etc.
2. **Retail Applications:** Person detection and tracking, cashier-less checkout systems, **tracking items in and out of a shopping cart**, tracking people leaving or entering the buildings, etc.
3. **Surveillance Cameras:** Crowd monitoring, **tracking mask usage**, tracking people with suspicious activities or items, etc.
4. **Sports Analytics: Tracking the trajectory of a basketball**, tracking football to

count the number of goals, tracking players to understand their movements and preventing injuries, etc.

5.2.2 Medical Image Analysis

What Is Computer Vision For Medical Imaging?

There is a technological boom in the healthcare industry and new advancements are being developed as we speak. Ever since the introduction of AI (artificial intelligence), healthcare has taken interest in how it can significantly assist professionals.

What Is Computer Vision And Medical Imaging?

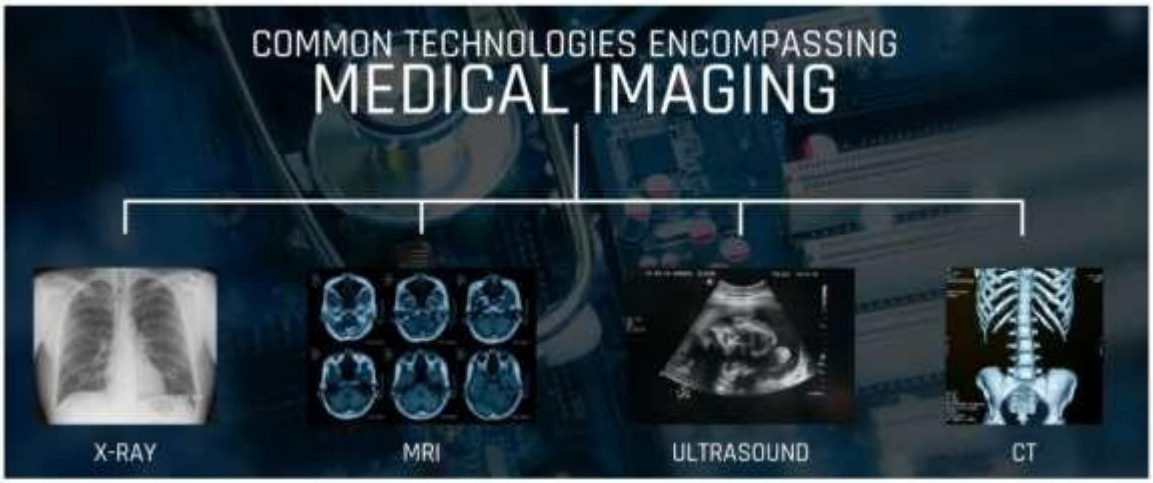


Figure 5.12 Medical Imaging

Computer Vision is an AI technology that enables computers to identify and recognize objects in images and videos. To help elaborate, ADAS is similar computer vision technology where cameras and sensors provide a live feed to an in-vehicle computer that processes the data and gives drivers feedback on their surroundings to provide a much safer driving experience. Throughout the many years in development, computer vision has proven to reliably analyze and differentiate images with 90% accuracy. Because of this, the technology has been accepted into the healthcare industry to automate and drastically improve the success rates of treatments. Computer vision has been applied to the healthcare industry to enable Medical Imaging.

What Are The Different Types Of Medical Imaging?

Medical imaging is an umbrella to several similar technologies that uses non-

invasive imagery to take different parts of the patient's body via:

- X-ray
- Ultrasound
- Magnetic Response Imagine (MRI)
- Computed Tomography (CT)
- Tactile Imaging
- Endoscopy

It incorporates AI to provide insight into possible abnormalities, injuries, or issues. Medical Imaging gives professionals a more in-depth visualization of the potential issue for a more accurate diagnosis. Doctors have used medical imaging to help diagnose early detection of diseases, assist in decision-making processes, and monitor the patient's health.

Benefits Of Medical Imaging

Early Detection With AI

AI has become reliable and accurate to distinguish, recognize, and identify abnormalities in various applications. Medical Imaging integrates AI to identify early traces of diseases where the human eye cannot. By detecting these issues in a timely manner, an appropriate diagnosis can be made, and treatment can begin before it develops to something more serious. Medical Imaging is capable of detecting various abnormalities:

- Tumors
- Cancer
- Stroke
- Pneumonia
- COVI and more

Need to be detected as early as possible to increase the chances of a successful treatment. Because these changes are minute, professionals turn to AI to help identify the most minor abnormalities for a much rapid and efficient turnaround time.

Improved Insight Data Analysis For Decision Making

Doctors and surgeons need as much information as possible to provide an accurate diagnosis or procedure. Medical Imaging has aided healthcare professionals with detailed imagery and more reliable information to develop a much more accurate diagnosis. Misidentifying an issue can cause irreversible damage and consequences for both the patient and doctor. With more intricate data, the odds of these incidents occurring significantly decrease.

Monitoring

Medical Imaging not only provides early detection, but it is also capable of monitoring abnormalities. Patients that are recovering from treatment can be evaluated over time to ensure when they are fully recovered. Some patients with non-life-threatening conditions need to be monitored for practitioners to decide if an operation will be needed or instruct the patient to follow a routine that will ease their condition naturally.

Other Top Applications Of Computer Vision In Healthcare

- Fracture detection
- Blood loss measurement
- Remote patient rehabilitation
- Diabetic retinopathy

What Technologies Matter For Medical Imaging?

Automation

With Edge computing, certain tasks such as updating clinical records are time consuming and can now be automated. Computer vision not only assists medical professionals in detecting early signs of anomalies, but also significantly increases their productivity. Professionals can focus on specified locations that the AI has highlighted and develop a much faster diagnosis turnaround time. Now, they can focus on more important tasks at hand rather than tedious work that technology can handle reliably and quickly.

Training

AI do not stop learning after their deep-learning stage. Medical professionals consistently provide the AI with new data, and it will continue to improve upon its ever-growing database. AI can greatly improve with the use of that data using inferencing; making medical imaging more accurate and efficient.

Real-Time Data Analysis

Edge computing processes hundreds of images with AI to accurately and efficiently provide valuable information for medical professionals. AI Edge Inference Computers are purpose-built to deliver the necessary processing power to enable data-intensive applications like computer vision. Rather than using the Cloud, which is inefficient and costly in bandwidth, all data analysis can occur in real-time and on premises.

Figure 5.14 Real time data Analysis

Privacy/Data Security

Data security is a top priority especially in the healthcare sector. Edge computing localizes both data storage and processing power to ensure there is no vulnerability to cyberattacks, whereas in cloud computing, data is prone to breaches and unauthorized acquisition. TPM 2.0 further protects the data with encryption and a tamper-free boot protocol.

Reliability

Rugged industrial computers are designed to operate seamlessly in the harshest conditions whether it be in remote locations, within vehicles, or on factory floors. To ensure 24/7 reliability, our industrial computers are equipped with several technologies that prevent the computer from being unresponsive during mission-critical operations: ECC memory, watchdog timers, RAID configurability, and more.

5.2.3 Content-Based Image Retrieval

What Is Content-Based Image Retrieval (CBIR)?

Content-Based Image Retrieval (CBIR) is a way of retrieving images from a database.

In CBIR, a user specifies a query image and gets the images in the database similar to the query image. To find the most similar images, CBIR compares the content of the input image to the database images.

More specifically, **CBIR compares visual features such as shapes, colors, texture and spatial information and measures the similarity between the query image with the images in the database with respect to those features:**

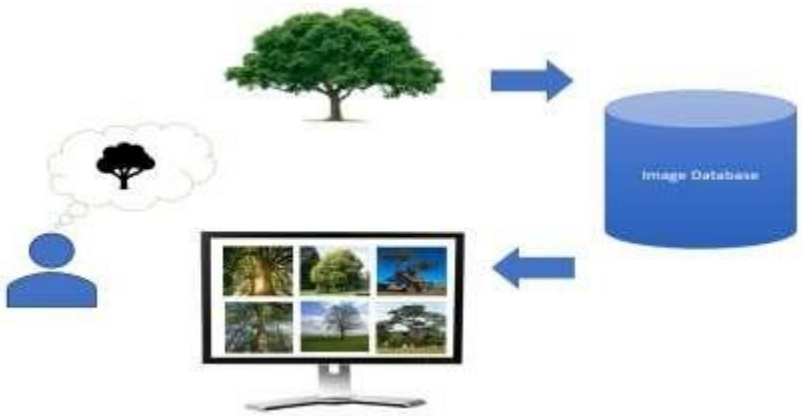


Figure 5.15 Content-Based Image Retrieval

In this picture, we have an example query image that illustrates the user's information need and a very large dataset of images. CBIR system's task is to rank all the images in the dataset according to how likely they are to fulfill the user's information need.

3. Difference Between Text-Based Image Retrieval and Content-Based Image Retrieval

In the Text-Based Image Retrieval (TBIR) approach, experts manually annotate

images with geolocation, keywords, tags, labels, or short descriptions. Users can use keywords, annotations, or descriptions to retrieve similar images that exist in the database.

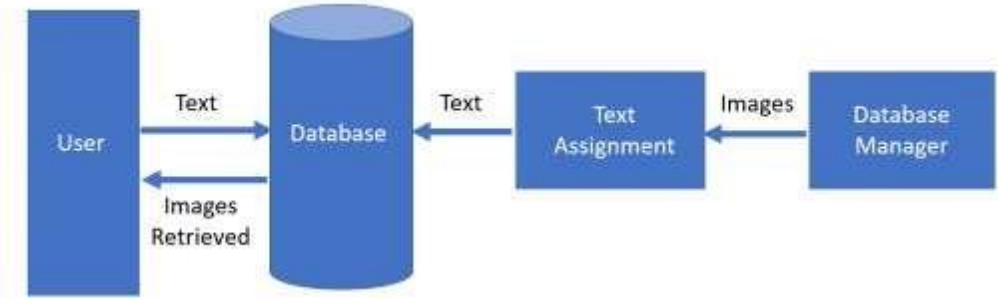


Figure 5.15 Difference Between Text-Based Image Retrieval and Content-Based Image Retrieval

As we see, **when users specify text or description as a search term, TBIR will retrieve the images that were assigned similar textual tags.**

The approach is simple and intuitive but has some disadvantages. Firstly, since it takes a considerable amount of time to annotate images manually, it's labor-intensive. Further, the tags may be unreliable due to the dependency on people's perceptions and interpretations that can vary widely across groups of individuals. To address those problems, CBIR compares the visual contents of images directly. In that way, there is no need for human labor as well as their subjective and error-prone perception.

4. Feature Extraction Methods in CBIR

CBIR systems need to perform feature extraction, which plays a significant role in representing an image's semantic content.

There are two main categories of visual features: Global and local.

1. Global Features

Global features are those that describe an entire image. They contain information on the entire image. For example, several descriptors characterize color spaces, such as color moments, color histograms, and so on.

Other global features are concerned with other visual elements such as e.g. shapes and

texture.

In this diagram, we find various methods for global feature extraction:

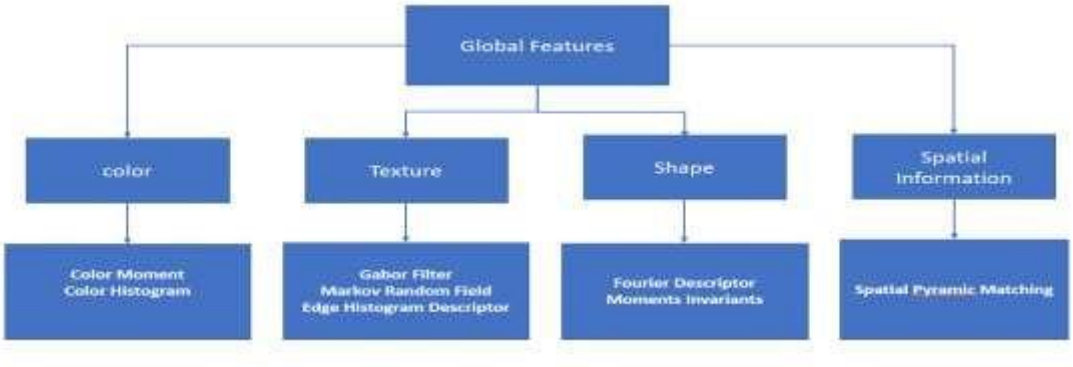


Figure 5.16 Global features are those that describe an entire image

5.3. Local Features

While global features have many advantages, they change under scaling and rotation. For this reason, local features are more reliable in various conditions.

Local features describe visual patterns or structures identifiable in small groups of pixels.

For example, edges, points, and various image patches.

The descriptors used to extract local features consider the regions centered around the detected visual structures. Those descriptors transform a local pixel neighborhood into a vector presentation.

One of the most used local descriptors is SIFT which stands for Scale-Invariant Feature Transform. It consists of a descriptor and a detector for key points. It doesn't change when we rotate the image we're working on. However, it has some drawbacks, such as needing a fixed vector for encoding and a huge amount of memory.

5. Deep Neural Networks

Recently, state-of-the-art CBIR systems have started using machine-learning methods such as deep-learning algorithms. They can perform feature extraction far better than traditional methods.

Usually, a Deep Convolutional Neural Network (DCNN) is trained using available data. Its job is to extract features from images. So, **when a user sends the query image to the database system, DCNN extracts its features.** Then, the query-image features are compared to those of the database images. In that step, the database system finds the most similar images using similarity measures and returns them to the user:

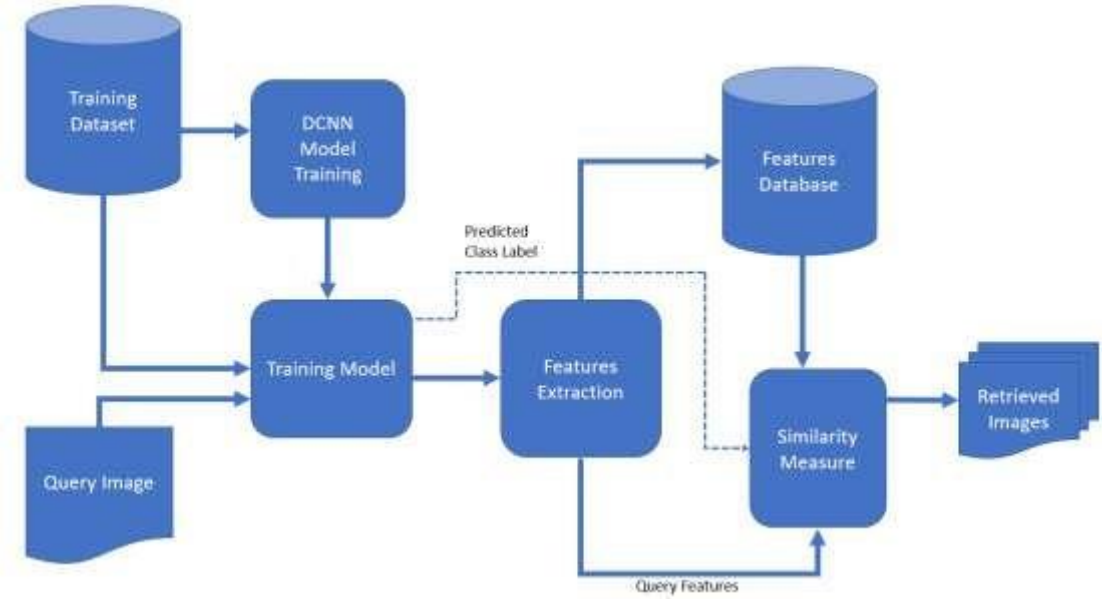


Figure 5.17 Deep Neural Networks

Since there are various pre-trained convolutional networks as well as Computer Vision Datasets, some people prefer ready-to-use models such as Alex Net, GoogLe Net, and ResNet50 over training their networks from scratch.

So, deep-learning models such as DCNN extract features automatically. In contrast, in traditional models, we pre-define the features to extract.

6. Similarity Measures

Similarity measures quantify how similar a database image is to our query image. The selection of the right similarity measure has always been a challenging task.

The structure of feature vectors drives the choice of the similarity measure.

There are two types of similarity measures: distance measures and similarity metrics.

5.4 Distance

A distance measure typically quantifies the dissimilarity of two feature vectors. We calculate it as the distance between two vectors in some metric space.

From there, the most similar images to the input image are those with the smallest distances to it.

There are several distance functions we can use. Manhattan distance, Mahalanobis distance, and Histogram Intersection Distance (HID) are some examples of distance measure functions.

Geometrically, HID is the intersection of two histograms:

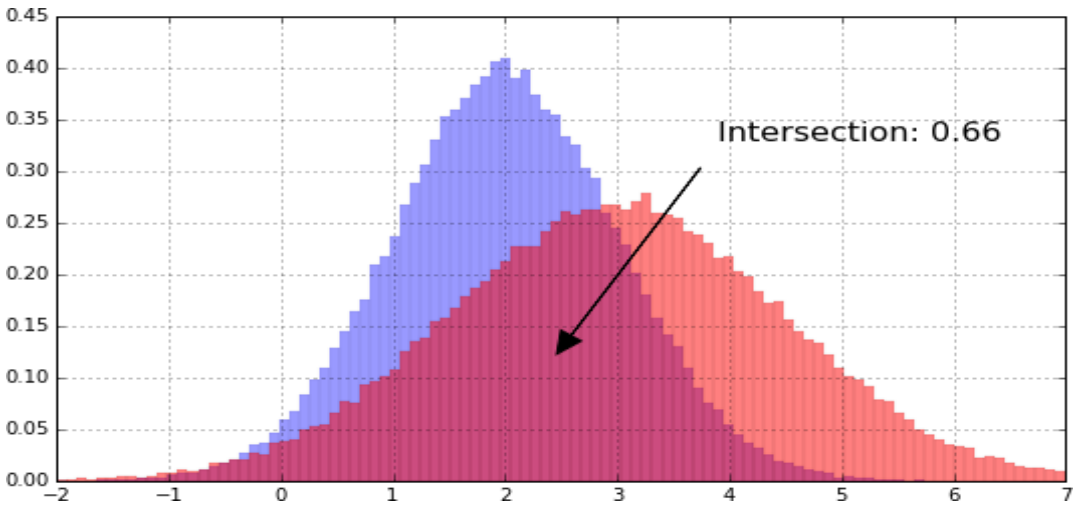


Figure 5.18 Geometrically, HID is the intersection of two histograms

5.5. Similarity Metrics

On the other hand, **a similarity metric quantifies the similarity between two feature vectors**. So, it works the opposite way to distance metrics: the most significant value shows the image similar to the query image.

For example, the cosine distance measures the angle between two feature vectors.

Given two vectors of features, *X* and *Y:*

$$CosineDistance(X, Y) = \frac{\langle X, Y \rangle}{|X| \times |Y|}$$

5.5.1. Video Data Processing

Video data processing in computer vision involves analyzing and extracting meaningful information from video sequences. This field has gained significant attention due to its wide range of applications, including surveillance, autonomous vehicles, human-computer interaction, and more. Here are key concepts and techniques involved in video data processing in computer vision:

1. **Frame-by-Frame Analysis:**

   Videos are typically composed of a sequence of frames. Analyzing each frame individually is a fundamental step. Techniques such as object detection, image segmentation, and feature extraction can be applied frame by frame.

2. **Motion Detection and Tracking:**

   Motion detection algorithms identify changes in consecutive frames, helping to detect moving objects. Object tracking methods then follow the movement of these objects across frames. Kalman filters, Mean-Shift, and correlation-based tracking are common techniques.

3. **Optical Flow:**

   Optical flow algorithms estimate the motion of objects between consecutive frames. They provide information about the direction and speed of pixel movements. Optical flow is crucial for understanding dynamic scenes and tracking moving objects.

4. **Background Subtraction:**

   Background subtraction techniques help identify foreground objects by distinguishing them from the static background. This is useful in scenarios where the camera is stationary, and only moving objects are of interest.

5. **Event Detection:**

   Event detection involves recognizing specific occurrences or patterns within the video sequence. This can include detecting abnormal behavior, identifying specific actions, or recognizing gestures.

6. **Feature Extraction:**

   Extracting relevant features from video frames is essential for tasks like object recognition and tracking. Deep learning approaches, such as Convolutional Neural Networks (CNNs), are commonly used for feature extraction in video data.

7. **Action Recognition:**

   Action recognition involves identifying and categorizing human activities or interactions within the video. Recurrent Neural Networks (RNNs) and 3D Convolutional Neural Networks (3D CNNs) are often employed for this task.

8. **Video Summarization:**

   Video summarization techniques aim to condense lengthy videos into shorter representations while retaining essential information. This is beneficial for storage, retrieval, and quick analysis.

9. **Temporal Convolutional Networks (TCNs):**

   TCNs are used to capture temporal dependencies in video data. They are effective in recognizing patterns and relationships between frames over time.

10. **Deep Learning Architectures:**

    Deep learning architectures, such as 3D CNNs, Long Short-Term Memory networks (LSTMs), and their combinations, have shown significant success in video data processing tasks. They can capture both spatial and temporal information.

11. Real-Time Processing:

Achieving real-time video processing is crucial in applications like surveillance and robotics. Optimizations, parallelization, and hardware acceleration (e.g., GPUs, TPUs) are often employed to enhance processing speed.

The combination of these techniques allows computer vision systems to understand, interpret, and respond to the dynamic nature of video data, enabling a wide range of applications in various industries.

5.5.2. Virtual Reality and Augmented Reality

5.5.2.1 Virtual Reality

Virtual Reality (VR) and Augmented Reality (AR) are immersive technologies that enhance the way we perceive and interact with the digital and physical worlds. While they share some similarities, they also have distinct characteristics.

Definition:

VR creates a completely immersive digital environment that users can interact with.

It typically involves the use of a head-mounted display (HMD) to block out the real world and replace it with a simulated one.

Immersive Experience:

Users are fully immersed in a computer-generated environment, often feeling as though they are physically present in that space.

Applications:

- Entertainment (gaming, virtual tourism, movies)
- Training and simulations (military, medical, aviation)
- Education (virtual classrooms, historical recreations)

Challenges:

- Motion sickness can be a concern for some users.
- High-quality VR experiences often require powerful hardware.

Understanding virtual reality and augmented reality

Figure 5.19 Understanding virtual reality and augmented reality

We spend a lot of time looking at screens these days. Computers, smartphones, and televisions have all become a big part of our lives; they're how we get a lot of our news, use social media, watch movies, and much more. **Virtual reality (VR)** and **augmented reality (AR)** are two technologies that are changing the way we use screens, creating new and exciting interactive experiences.

Virtual reality uses a headset to place you in **a computer-generated world** that you can explore. **Augmented reality**, on the other hand, is a bit different. Instead of transporting you to a virtual world, it **takes digital images and layers them on the real world around you** through the use of either a clear visor or smartphone.

With virtual reality, you could explore an underwater environment. With augmented reality, you could see fish swimming through the world around you.

Watch the video below to learn about virtual reality and augmented reality.

Virtual reality

Virtual reality immerses you in a virtual world through the use of a **headset** with

some type of screen displaying a virtual environment. These headsets also use a technology called **head tracking,** which allows you to look around the environment by **physically moving your head**. The display will follow whichever direction you move, giving you a 360-degree view of the virtual environment.

Types of VR devices

At the moment, there are two major types of headsets. Both have their pros and cons, which you'll want to consider if you're looking to purchase one.

The first type has a **screen built in to the headset.** These devices connect to a computer and require a pretty powerful system to operate smoothly. They have great graphics and perform well, but they're also **pretty expensive.** A few popular examples of these include the Oculus Rift, the Vive, and the PlayStation VR, which connects to the PlayStation 4 game console.

Some of these devices come with **handheld controllers** that track your hands' movements as well, providing for a more interactive experience.

The other type of headset **houses your phone and uses its screen as the display.** These don't require a computer and run completely off of apps on your smartphone. The graphics and performance levels on these headsets are n't quite as good as those with a built-in screen, but they do tend to be **much cheaper.** Some popular examples include Google Cardboard and the Gear VR.

5.5.2.2 Augmented reality

Definition:

AR overlays digital information onto the real-world environment, enhancing the user's perception of the surroundings.

Integration with Reality:

Users can still see and interact with the real world while digital information is superimposed on it.

Applications:

- Mobile applications (Pokemon Go, Snapchat filters)
- Industrial use (maintenance and repair guidance)
- Healthcare (surgical assistance, medical imaging)

Challenges:

- Seamless integration of digital and real-world elements can be challenging.
- Limited field of view and hardware constraints in some devices.

Augmented reality allows you to **see the world around you with digital images layered on top of it.** There are currently a couple of AR headsets available, including the Microsoft HoloLens and the Magic Leap. However, they are currently more expensive than VR headsets, and are marketed primarily to businesses.

Figure 5.22 Augmented reality

Figure 5.23 AR using filters on Snapchat.

Augmented reality can also be used on devices like **smartphones** and **laptops** without the use of a headset. There are a variety of apps that use AR, including some that allow you to **translate text using your camera, identify stars** in the sky, and even **see how your garden would look with different plants**. You may have even previously used AR without realizing it, while playing a game like **Pokemon Go** or using **filters on Snapchat**.

5.5.2.3 Mixed Reality (MR):

Definition:

Mixed Reality combines aspects of both VR and AR, allowing digital and physical elements to interact in real-time.

Interaction:

MR enables users to interact with both virtual and real-world objects simultaneously.

Applications:

- Design and engineering (3D modeling)
- Collaboration and communication (virtual meetings with shared holograms)

Challenges:

- Achieving seamless integration of virtual and real-world elements.
- Hardware requirements for high-quality MR experiences.

Common Technologies:

Headsets:

VR and AR often use specialized headsets, like Oculus Rift for VR and Microsoft HoloLens for AR.

Sensors:

Both VR and AR devices use sensors to track the user's movements and adjust the digital content accordingly.

Input Devices:

Controllers, gloves, and other input devices are used to interact with the virtual or augmented environment.

Content Creation:

Creating content for VR and AR involves specialized tools and software, including 3D modeling and spatial computing.